

Artificial Neural Networks and AI Concepts

Dr. Innocent Maposa

Division of Epidemiology & Biostatistics
Department of Global Health
Stellenbosch University

June 22, 2026

*AFRICA DATA SCIENCE CONFERENCE WORKSHOP: Machine Learning and Artificial
Intelligence in Healthcare*

- 1 AI Overview
 - Intelligence
 - AI landscape
 - ML Milestones
- 2 Foundations of AI
- 3 Artificial Neural Networks
 - Learning and loss function
- 4 Deep Learning
- 5 Transfer Learning
- 6 PAMHOYA Discovery Platform

What is Intelligence?

The Core Question

Intelligence has no single agreed definition.

Generally define: as the ability to **perceive**, **reason**, **learn**, and **act** in ways that achieve goals in complex environments.

Four ingredients of intelligence:

1

Perception

Sensing and interpreting the environment

2

Reasoning

Drawing inferences from information

3

Learning

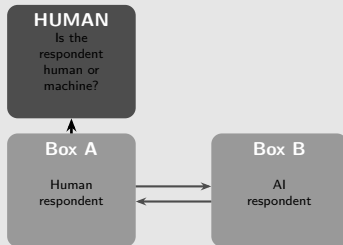
Improving from experience

4

Action

Producing goal-directed behaviour

The Turing Test (1950)



If the human cannot distinguish,
the machine passes the test

"Can machines think?"

— Alan Turing (1950)

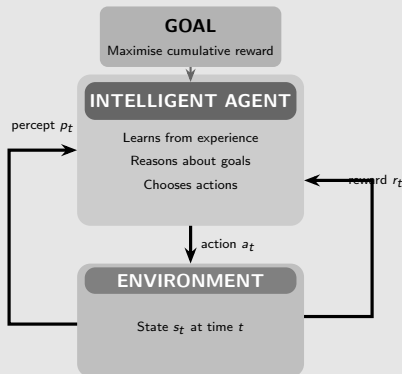
Definition 1 — Behavioural

AI is the study and design of **intelligent agents**: systems that perceive their environment through sensors and take actions that **maximise their chance of achieving goals** [9].

Definition 2

AI encompasses any computational technique that enables machines to **exhibit behaviours associated with human cognitive functions**—learning, reasoning, problem-solving, perception, and language use [6].

The Intelligent Agent Model



Artificial Intelligence

Any technique that enables machines to **mimic human cognitive function**. Includes rule-based, symbolic, and statistical approaches.

Machine Learning

A **subset of AI** in which systems learn from data. Shock: "We do not program the rules; we show examples and let the machine find them."

Deep Learning

A **subset of ML** using neural networks with many layers. Excels at perceptual tasks: images, text, audio.

Foundation Models / LLMs

Large-scale deep learning models trained on internet-scale data; underlie ChatGPT, Claude, Gemini.

The Three Levels of AI Capability [2]

NARROW AI (ANI)

Current Reality
Excels at **one specific task**.
Cannot generalise beyond
its training domain.

Examples:
Chess engines, image classifiers,
speech recognition, ChatGPT,
recommendation systems.

“The Specialist”

GENERAL AI (AGI)

Hypothetical / Emerging
Performs **any cognitive task**
that a human can.

Transfers knowledge between
completely different domains.

Status:
No confirmed AGI exists.
Active research area.
Estimated: decades away.

“The Generalist”

SUPERINTELLIGENCE

Theoretical / Speculative
Surpasses **all human intelligence**
across every domain.

Could improve itself recursively
(*intelligence explosion*).

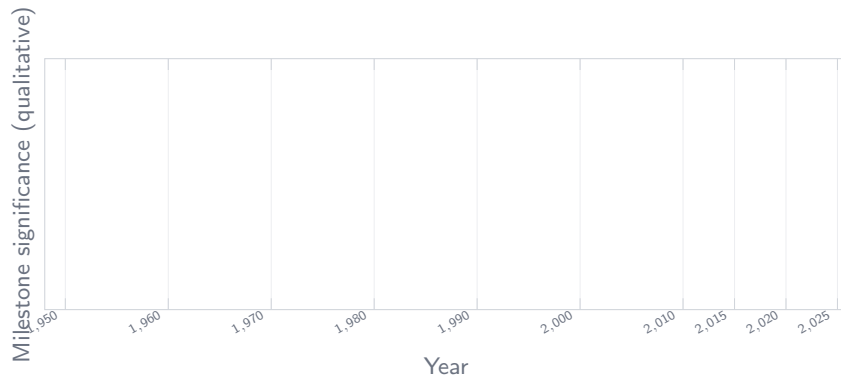
Status:
Does not exist.
Central to AI safety research
(Bostrom, Russell).

“The Singularity”

NOW

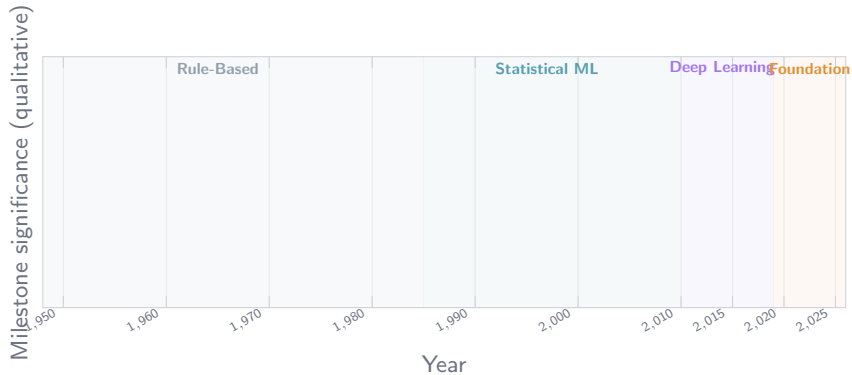
ML Major Milestones

Seven decades of progress



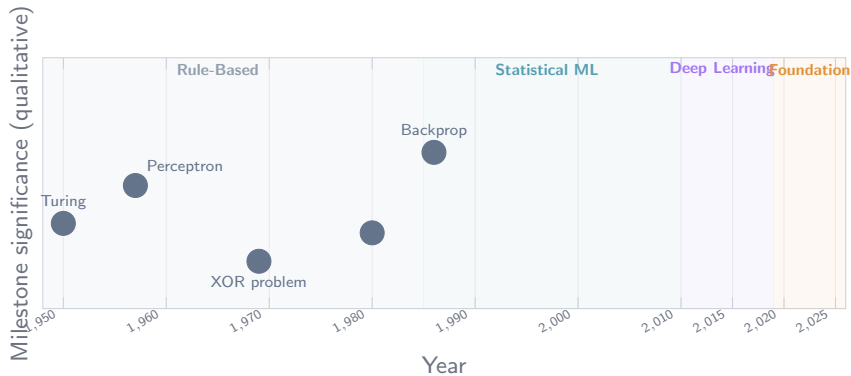
ML Major Milestones

Seven decades of progress



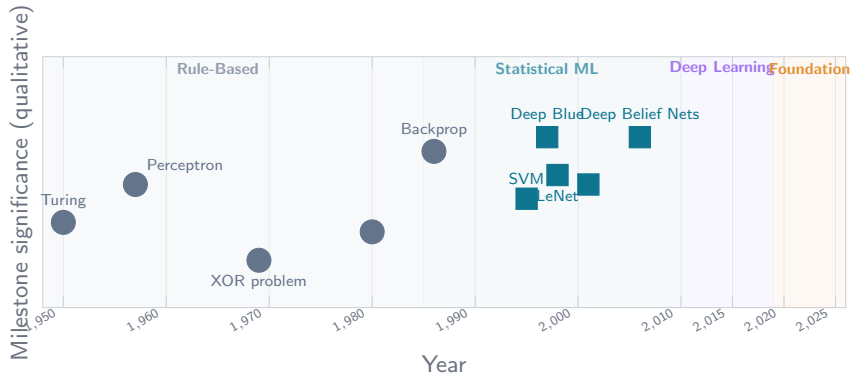
ML Major Milestones

Seven decades of progress



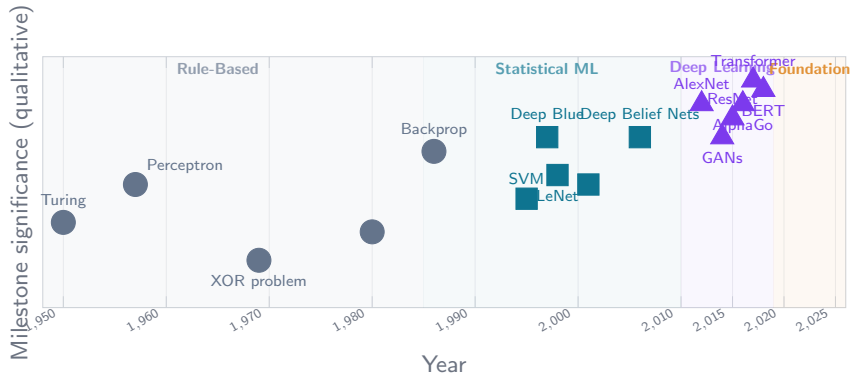
ML Major Milestones

Seven decades of progress



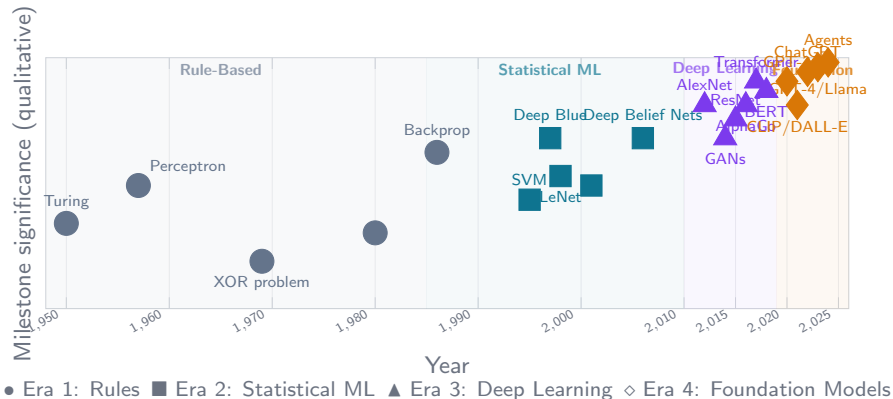
ML Major Milestones

Seven decades of progress



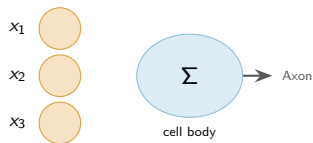
ML Major Milestones

Seven decades of progress

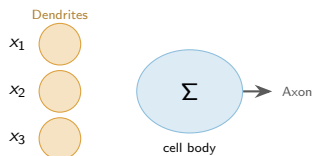


- 1 AI Overview
 - Intelligence
 - AI landscape
 - ML Milestones
- 2 Foundations of AI
- 3 Artificial Neural Networks
 - Learning and loss function
- 4 Deep Learning
- 5 Transfer Learning
- 6 PAMHOYA Discovery Platform

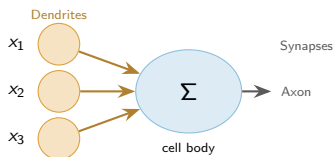
- The brain contains
 $\approx$ **86 billion** neurons



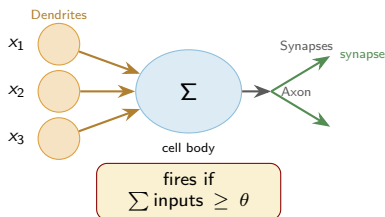
- The brain contains
 $\approx$ **86 billion** neurons
- Each neuron connects to up to
10,000 others



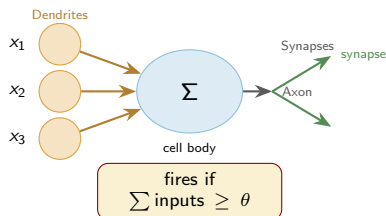
- The brain contains $\approx$ **86 billion** neurons
- Each neuron connects to up to **10,000** others
- Neurons **receive**, **integrate**, and **transmit** signals



- The brain contains $\approx$ **86 billion** neurons
- Each neuron connects to up to **10,000** others
- Neurons **receive**, **integrate**, and **transmit** signals
- Fires when input exceeds a **threshold θ**



- The brain contains $\approx$ **86 billion** neurons
 - Each neuron connects to up to **10,000** others
 - Neurons **receive**, **integrate**, and **transmit** signals
 - Fires when input exceeds a **threshold θ**
 - Learning = **strengthening / weakening** synapses
- "Neurons that fire together, wire together"*



- 1 AI Overview
 - Intelligence
 - AI landscape
 - ML Milestones
- 2 Foundations of AI
- 3 Artificial Neural Networks
 - Learning and loss function
- 4 Deep Learning
- 5 Transfer Learning
- 6 PAMHOYA Discovery Platform

The Artificial Neuron (Perceptron)

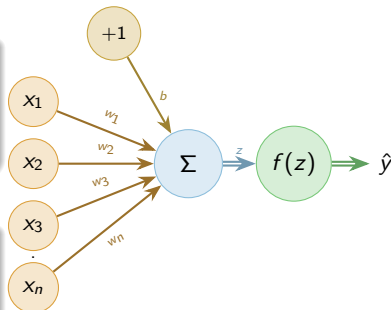
Weighted Sum

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

Output

$$\hat{y} = f(z)$$

where f is an **activation function**



Key Insight

Learning = adjusting weights w_j and bias b .

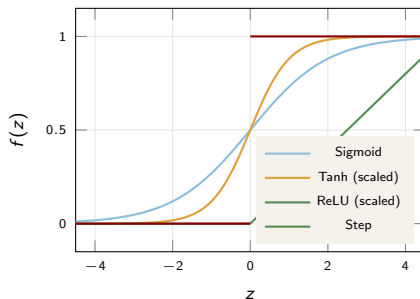
Components:

- x_i — inputs (features)
- w_i — **weights** (learned)
- b — bias (shifts boundary)
- $f(\cdot)$ — non-linearity

Why are they needed?

- Without activation, any depth network collapses to a **linear model**
- Non-linearity $\Rightarrow$ learn complex patterns
- Choice of f affects: gradient flow, training speed, output range

Common Activation Functions



Key Property

A non-linear activation gives neural networks their **expressive power**.

Activation Functions cont..

Function	Formula	Range	Pros	Cons
Sigmoid	$\frac{1}{1+e^{-z}}$	$(0, 1)$	Probabilistic	Vanishing grad.
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	Zero-centred	Vanishing grad.
ReLU	$\max(0, z)$	$[0, \infty)$	Fast, sparse	Dying neurons
Leaky ReLU	$\max(\alpha z, z)$	$(-\infty, \infty)$	Fixes dying ReLU	Extra α
Softmax	$\frac{e^{z_j}}{\sum_k e^{z_k}}$	$(0, 1)^K$	Multi-class	Output only

Tip

ReLU and its variants are the default choice for hidden layers in modern deep networks.

Case study: Logistic vs ANN

Data

- 8 predictors and diabetes outcome (binary)
- $n=768$
- split: train=538, test=230
- ANN 1 hidden layer & 5 neurons

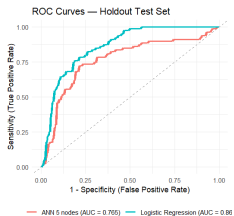
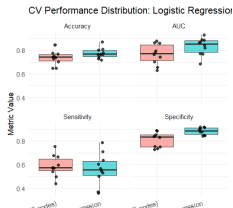


Figure: Logistic vs ANN

10-Fold CV Performance by Fold



Figure: Compare Performance



ANN model

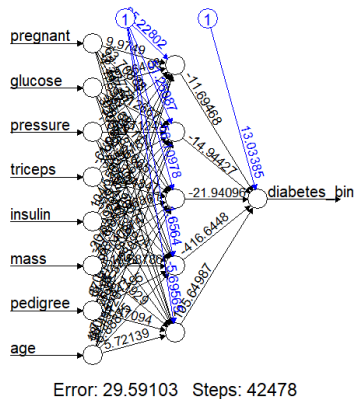


Figure: Fitted ANN model

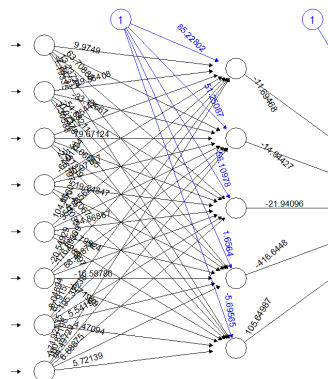
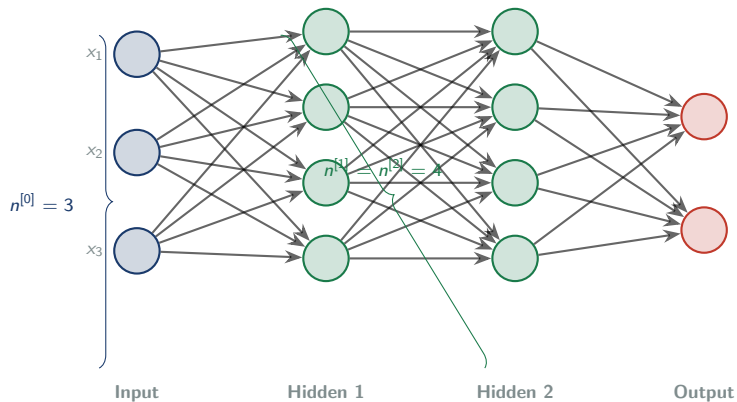


Figure: Zoomed out weights

Neural Network Architecture



Vectorized layers

Single hidden layer, vectorized:

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

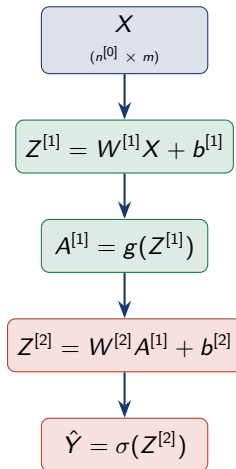
$$A^{[1]} = g^{[1]}(Z^{[1]})$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$\hat{Y} = A^{[2]} = g^{[2]}(Z^{[2]})$$

Dimensions for m examples, $n^{[0]}$ inputs, $n^{[1]}$ hidden:

- $W^{[1]} : n^{[1]} \times n^{[0]}$
- $b^{[1]} : n^{[1]} \times 1$ (broadcast)
- $A^{[1]} : n^{[1]} \times m$ (m batches)
- $W^{[2]} : n^{[2]} \times n^{[1]}$



Loss function

Binary Cross-Entropy Loss:

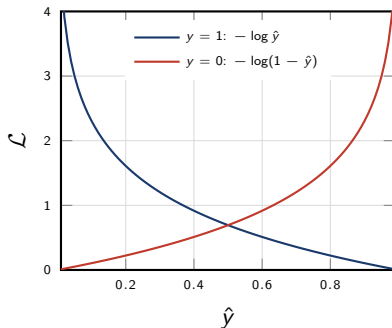
$$\mathcal{L}(\hat{y}, y) = -[y \log \hat{y} + (1-y) \log(1-\hat{y})]$$

Cost over m examples:

$$J(W, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Goal

Find $W^*, b^* = \arg \min_{W, b} J(W, b)$



Parameter update rule:

$$W \leftarrow W - \eta \frac{\partial J}{\partial W}, \quad b \leftarrow b - \eta \frac{\partial J}{\partial b}$$

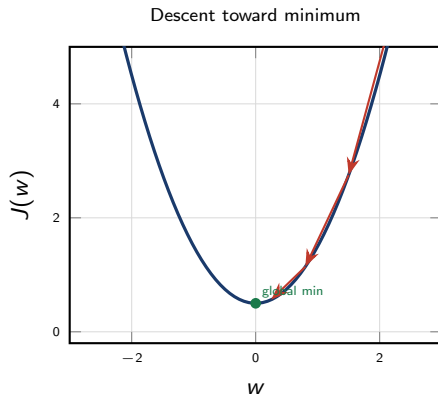
- η : learning rate (hyperparameter)
- Repeat until convergence

Variants:

- **Batch GD**: all m examples
- **Stochastic GD**: 1 example
- **Mini-batch**: size B (e.g. 32, 64, 128, 256)

Tip

Mini-batch $B \in \{32, \dots, 512\}$ is the sweet spot for GPU utilization.



Chain rule through computation:

$$dZ^{[2]} = A^{[2]} - Y$$

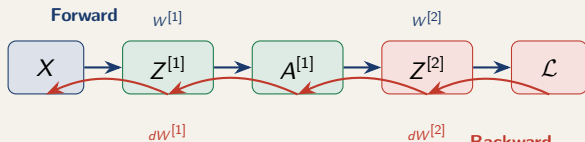
$$dW^{[2]} = \frac{1}{m} dZ^{[2]} (A^{[1]})^\top$$

$$db^{[2]} = \frac{1}{m} \sum dZ^{[2]}$$

$$dZ^{[1]} = (W^{[2]})^\top dZ^{[2]} \odot g'^{[1]}(Z^{[1]})$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^\top$$

$$db^{[1]} = \frac{1}{m} \sum dZ^{[1]}$$



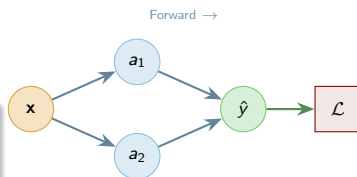
Goal: compute $\frac{\partial \mathcal{L}}{\partial w}$ for every weight.

Chain Rule

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Algorithm (3 steps):

- 1 **Forward pass** — compute $\hat{y}$ and $\mathcal{L}$



Efficiency

Backprop computes **all** gradients in just **two** passes.

Backpropagation — The Chain Rule

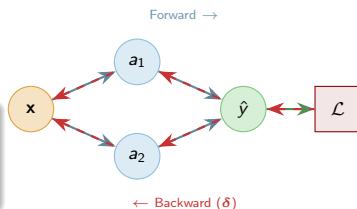
Goal: compute $\frac{\partial \mathcal{L}}{\partial w}$ for every weight.

Chain Rule

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Algorithm (3 steps):

- 1 **Forward pass** — compute $\hat{y}$ and $\mathcal{L}$
- 2 **Backward pass** — propagate δ from output to input



$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial z^{(L)}} \quad \delta^{(\ell)} = (w^{(\ell+1)})^\top \delta^{(\ell+1)} \odot f'(z^{(\ell)})$$

Efficiency

Backprop computes **all** gradients in just **two** passes.

Backpropagation — The Chain Rule

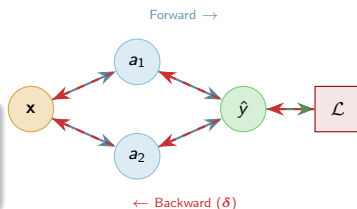
Goal: compute $\frac{\partial \mathcal{L}}{\partial w}$ for every weight.

Chain Rule

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Algorithm (3 steps):

- 1 **Forward pass** — compute $\hat{y}$ and $\mathcal{L}$
- 2 **Backward pass** — propagate δ from output to input
- 3 **Update** — $w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$



$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial z^{(L)}} \quad \delta^{(\ell)} = (w^{(\ell+1)})^\top \delta^{(\ell+1)} \odot f'(z^{(\ell)})$$

Efficiency

Backprop computes **all** gradients in just **two** passes.

Architecture:

- Inputs: i_1, i_2
- Hidden: h_1, h_2
- Outputs: o_1, o_2
- Biases: b_1, b_2
- Activation: **sigmoid** σ

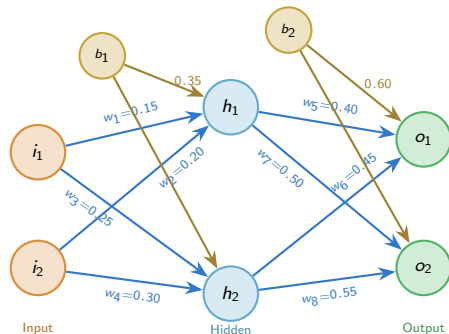
Training pair

Inputs: $i_1=0.05, i_2=0.10$

Targets: $t_1=0.01, t_2=0.99$

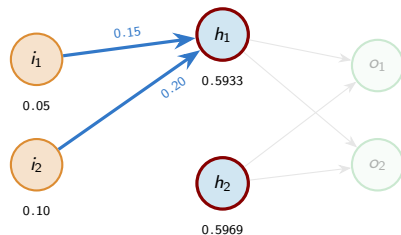
Biases & learning rate

$b_1=0.35, b_2=0.60, \eta=0.5$



Net input to h_1

$$\begin{aligned}\text{net}_{h_1} &= w_1 i_1 + w_2 i_2 + b_1 \\ &= 0.15(0.05) + 0.20(0.10) + 0.35 \\ &= \mathbf{0.3775}\end{aligned}$$

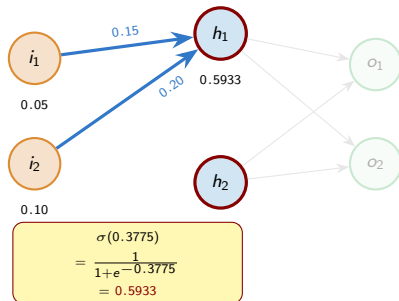


Net input to h_1

$$\begin{aligned}\text{net}_{h_1} &= w_1 i_1 + w_2 i_2 + b_1 \\ &= 0.15(0.05) + 0.20(0.10) + 0.35 \\ &= 0.3775\end{aligned}$$

Sigmoid activation at h_1

$$\text{out}_{h_1} = \sigma(0.3775) = \frac{1}{1+e^{-0.3775}} = 0.5933$$



Net input to h_1

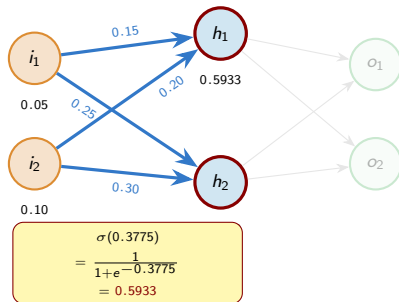
$$\begin{aligned}\text{net}_{h_1} &= w_1 i_1 + w_2 i_2 + b_1 \\ &= 0.15(0.05) + 0.20(0.10) + 0.35 \\ &= 0.3775\end{aligned}$$

Sigmoid activation at h_1

$$\text{out}_{h_1} = \sigma(0.3775) = \frac{1}{1+e^{-0.3775}} = 0.5933$$

Repeat for h_2

$$\begin{aligned}\text{net}_{h_2} &= 0.25(0.05) + 0.30(0.10) + 0.35 = 0.3925 \\ \text{out}_{h_2} &= \sigma(0.3925) = 0.5969\end{aligned}$$



Net input to h_1

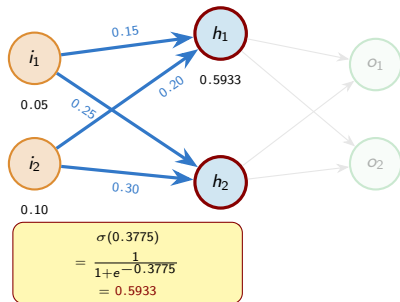
$$\begin{aligned}\text{net}_{h_1} &= w_1 i_1 + w_2 i_2 + b_1 \\ &= 0.15(0.05) + 0.20(0.10) + 0.35 \\ &= 0.3775\end{aligned}$$

Sigmoid activation at h_1

$$\text{out}_{h_1} = \sigma(0.3775) = \frac{1}{1+e^{-0.3775}} = 0.5933$$

Repeat for h_2

$$\begin{aligned}\text{net}_{h_2} &= 0.25(0.05) + 0.30(0.10) + 0.35 = 0.3925 \\ \text{out}_{h_2} &= \sigma(0.3925) = 0.5969\end{aligned}$$

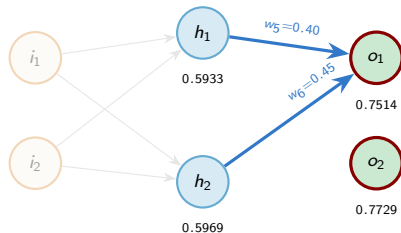


Sigmoid & derivative

$$\sigma(z) = \frac{1}{1+e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Net input to o_1

$$\begin{aligned}\text{net}_{o_1} &= w_5 \text{out}_{h_1} + w_6 \text{out}_{h_2} + b_2 \\ &= 0.40(0.5933) + 0.45(0.5969) + 0.60 \\ &= \mathbf{1.1059}\end{aligned}$$

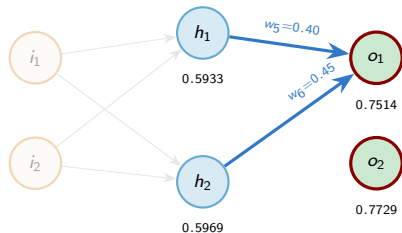


Net input to o_1

$$\begin{aligned}\text{net}_{o_1} &= w_5 \text{out}_{h_1} + w_6 \text{out}_{h_2} + b_2 \\ &= 0.40(0.5933) + 0.45(0.5969) + 0.60 \\ &= 1.1059\end{aligned}$$

Output of o_1

$$\text{out}_{o_1} = \sigma(1.1059) = \frac{1}{1+e^{-1.1059}} = 0.7514$$



Forward Pass — Output Layer

Net input to o_1

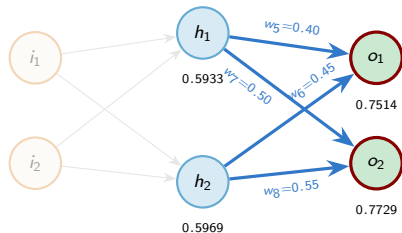
$$\begin{aligned}\text{net}_{o_1} &= w_5 \text{out}_{h_1} + w_6 \text{out}_{h_2} + b_2 \\ &= 0.40(0.5933) + 0.45(0.5969) + 0.60 \\ &= \mathbf{1.1059}\end{aligned}$$

Output of o_1

$$\text{out}_{o_1} = \sigma(1.1059) = \frac{1}{1+e^{-1.1059}} = \mathbf{0.7514}$$

Repeat for o_2

$$\begin{aligned}\text{net}_{o_2} &= 0.50(0.5933) + 0.55(0.5969) + 0.60 = \mathbf{1.2245} \\ \text{out}_{o_2} &= \sigma(1.2245) = \mathbf{0.7729}\end{aligned}$$



Forward Pass — Output Layer

Net input to o_1

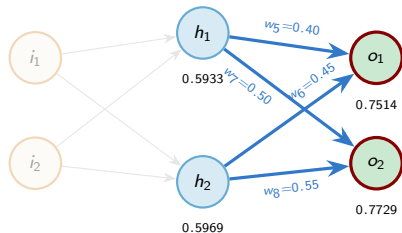
$$\begin{aligned}\text{net}_{o_1} &= w_5 \text{out}_{h_1} + w_6 \text{out}_{h_2} + b_2 \\ &= 0.40(0.5933) + 0.45(0.5969) + 0.60 \\ &= \mathbf{1.1059}\end{aligned}$$

Output of o_1

$$\text{out}_{o_1} = \sigma(1.1059) = \frac{1}{1+e^{-1.1059}} = \mathbf{0.7514}$$

Repeat for o_2

$$\begin{aligned}\text{net}_{o_2} &= 0.50(0.5933) + 0.55(0.5969) + 0.60 = \mathbf{1.2245} \\ \text{out}_{o_2} &= \sigma(1.2245) = \mathbf{0.7729}\end{aligned}$$



Prediction vs. Target

	Predicted	Target
o_1 :	0.7514	0.01 (far off!)
o_2 :	0.7729	0.99 (too low)

Total Error Calculation

Squared-error loss function

$$E = \sum_k \frac{1}{2} (t_k - \hat{y}_k)^2$$

Error at o_1 (target = 0.01)

$$E_{o_1} = \frac{1}{2} (0.01 - 0.7514)^2 = \frac{1}{2} (0.7414)^2 = 0.2748$$

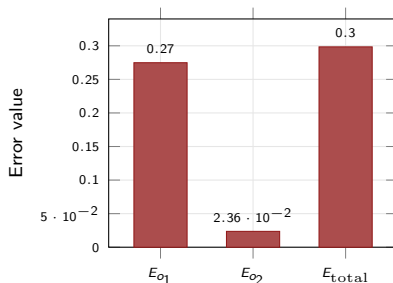
Error at o_2 (target = 0.99)

$$E_{o_2} = \frac{1}{2} (0.99 - 0.7729)^2 = \frac{1}{2} (0.2171)^2 = 0.0236$$

Total network error

$$E_{\text{total}} = E_{o_1} + E_{o_2} = 0.2748 + 0.0236 = 0.2984$$

Error decomposition



Why $\frac{1}{2}$?

$\frac{d}{d\hat{y}} \frac{1}{2} (t - \hat{y})^2 = -(t - \hat{y})$
The exponent cancels cleanly.

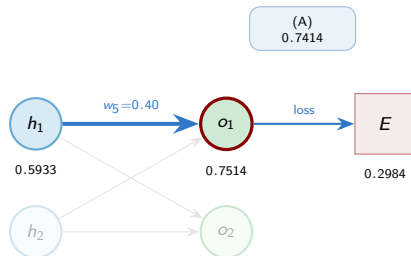
Goal: $\frac{\partial E_{\text{total}}}{\partial w_5}$

Three-factor chain rule:

$$\frac{\partial E_{\text{total}}}{\partial w_5} = \underbrace{\frac{\partial E}{\partial \text{out}_{o_1}}}_{(A)} \cdot \underbrace{\frac{\partial \text{out}_{o_1}}{\partial \text{net}_{o_1}}}_{(B)} \cdot \underbrace{\frac{\partial \text{net}_{o_1}}{\partial w_5}}_{(C)}$$

(A) Loss w.r.t. output

$$\begin{aligned}\frac{\partial E}{\partial \text{out}_{o_1}} &= -(t_1 - \text{out}_{o_1}) \\ &= -(0.01 - 0.7514) = \mathbf{0.7414}\end{aligned}$$



Goal: $\frac{\partial E_{\text{total}}}{\partial w_5}$

Three-factor chain rule:

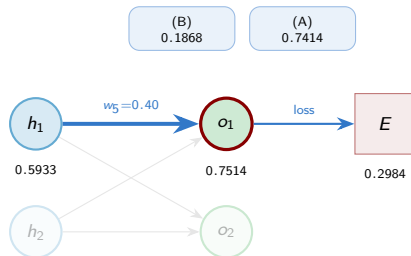
$$\frac{\partial E_{\text{total}}}{\partial w_5} = \underbrace{\frac{\partial E}{\partial \text{out}_{o_1}}}_{(A)} \cdot \underbrace{\frac{\partial \text{out}_{o_1}}{\partial \text{net}_{o_1}}}_{(B)} \cdot \underbrace{\frac{\partial \text{net}_{o_1}}{\partial w_5}}_{(C)}$$

(A) Loss w.r.t. output

$$\begin{aligned}\frac{\partial E}{\partial \text{out}_{o_1}} &= -(t_1 - \text{out}_{o_1}) \\ &= -(0.01 - 0.7514) = \mathbf{0.7414}\end{aligned}$$

(B) Sigmoid derivative

$$\frac{\partial \text{out}_{o_1}}{\partial \text{net}_{o_1}} = \text{out}_{o_1}(1 - \text{out}_{o_1})$$



Goal: $\frac{\partial E_{\text{total}}}{\partial w_5}$

Three-factor chain rule:

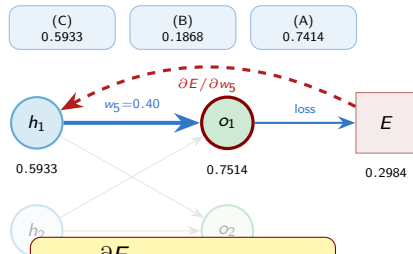
$$\frac{\partial E_{\text{total}}}{\partial w_5} = \underbrace{\frac{\partial E}{\partial \text{out}_{o_1}}}_{(A)} \cdot \underbrace{\frac{\partial \text{out}_{o_1}}{\partial \text{net}_{o_1}}}_{(B)} \cdot \underbrace{\frac{\partial \text{net}_{o_1}}{\partial w_5}}_{(C)}$$

(A) Loss w.r.t. output

$$\begin{aligned}\frac{\partial E}{\partial \text{out}_{o_1}} &= -(t_1 - \text{out}_{o_1}) \\ &= -(0.01 - 0.7514) = 0.7414\end{aligned}$$

(B) Sigmoid derivative

$$\frac{\partial \text{out}_{o_1}}{\partial \text{net}_{o_1}} = \text{out}_{o_1}(1 - \text{out}_{o_1})$$



$$\begin{aligned}\frac{\partial E}{\partial w_5} &= 0.7414 \times \\ &\quad 0.1868 \times 0.5933 \\ &= 0.08217\end{aligned}$$

(C) Net input w.r.t. weight

$$\frac{\partial \text{net}_{o_1}}{\partial w_5} = \text{out}_{h_1} = 0.5933$$

Gradient descent update ($\eta = 0.5$)

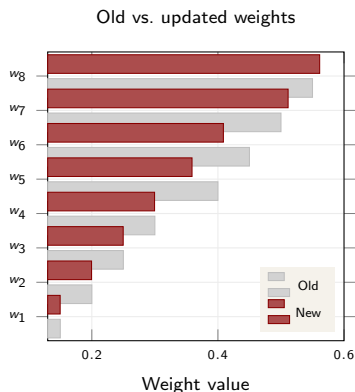
$$w^{\text{new}} = w - \eta \frac{\partial E_{\text{total}}}{\partial w}$$

Updating w_5

$$w_5^{\text{new}} = 0.40 - 0.5 \times 0.08217 = 0.3589$$

All output-layer weight updates:

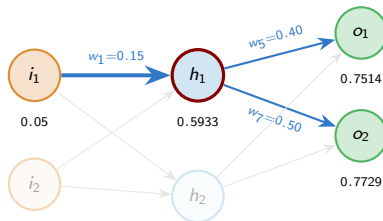
Weight	Old	Gradient	New
w_5	0.4000	+0.08217	0.3589
w_6	0.4500	+0.08266	0.4087
w_7	0.5000	-0.02260	0.5113
w_8	0.5500	-0.02274	0.5614



Goal: $\frac{\partial E_{\text{total}}}{\partial w_1}$

Error flows back through *both* outputs:

$$\frac{\partial E_{\text{total}}}{\partial \text{out}_{h_1}} = \frac{\partial E_{o_1}}{\partial \text{out}_{h_1}} + \frac{\partial E_{o_2}}{\partial \text{out}_{h_1}}$$



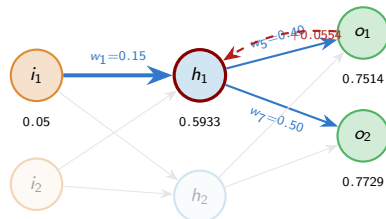
Goal: $\frac{\partial E_{\text{total}}}{\partial w_1}$

Error flows back through *both* outputs:

$$\frac{\partial E_{\text{total}}}{\partial \text{out}_{h_1}} = \frac{\partial E_{o_1}}{\partial \text{out}_{h_1}} + \frac{\partial E_{o_2}}{\partial \text{out}_{h_1}}$$

Via o_1

$$\begin{aligned} \frac{\partial E_{o_1}}{\partial \text{out}_{h_1}} &= (o_1 - t_1) \cdot o_1(1 - o_1) \cdot w_5 \\ &= 0.7414 \times 0.1868 \times 0.40 = \mathbf{0.05539} \end{aligned}$$

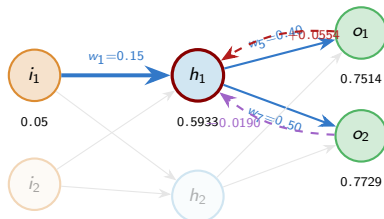


Backward Pass — Hidden Layer Weight w_1

Goal: $\frac{\partial E_{\text{total}}}{\partial w_1}$

Error flows back through *both* outputs:

$$\frac{\partial E_{\text{total}}}{\partial \text{out}_{h_1}} = \frac{\partial E_{o_1}}{\partial \text{out}_{h_1}} + \frac{\partial E_{o_2}}{\partial \text{out}_{h_1}}$$



Via o_1

$$\begin{aligned}\frac{\partial E_{o_1}}{\partial \text{out}_{h_1}} &= (o_1 - t_1) \cdot o_1(1 - o_1) \cdot w_5 \\ &= 0.7414 \times 0.1868 \times 0.40 = \mathbf{0.05539}\end{aligned}$$

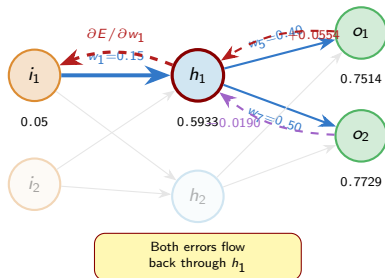
Via o_2

$$\frac{\partial E_{o_2}}{\partial \text{out}_{h_1}} = (o_2 - t_2) \cdot o_2(1 - o_2) \cdot w_7$$

Goal: $\frac{\partial E_{\text{total}}}{\partial w_1}$

Error flows back through *both* outputs:

$$\frac{\partial E_{\text{total}}}{\partial \text{out}_{h_1}} = \frac{\partial E_{o_1}}{\partial \text{out}_{h_1}} + \frac{\partial E_{o_2}}{\partial \text{out}_{h_1}}$$



Via o_1

$$\begin{aligned} \frac{\partial E_{o_1}}{\partial \text{out}_{h_1}} &= (o_1 - t_1) \cdot o_1(1 - o_1) \cdot w_5 \\ &= 0.7414 \times 0.1868 \times 0.40 = \mathbf{0.05539} \end{aligned}$$

Via o_2

$$\frac{\partial E_{o_2}}{\partial \text{out}_{h_1}} = (o_2 - t_2) \cdot o_2(1 - o_2) \cdot w_7$$

Full Gradient for w_1 & All Hidden-Layer Updates

Complete chain for w_1

$$\begin{aligned}\frac{\partial E}{\partial w_1} &= \underbrace{\frac{\partial E}{\partial \text{out}_{h_1}}}_{0.03634} \cdot \underbrace{h_1(1-h_1)}_{0.5933 \times 0.4067} \cdot \underbrace{i_1}_{0.05} \\ &= 0.03634 \times 0.2413 \times 0.05 = \mathbf{0.000439}\end{aligned}$$

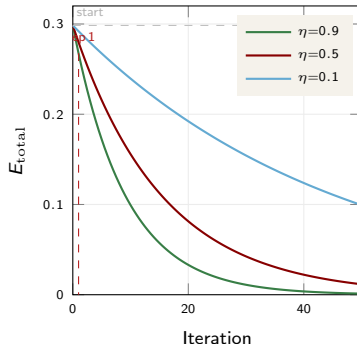
Update w_1 ($\eta = 0.5$)

$$w_1^{\text{new}} = 0.15 - 0.5 \times 0.000439 = \mathbf{0.14978}$$

All hidden-layer weight updates:

Wt	Old	Gradient	New
w_1	0.1500	4.39×10^{-4}	0.14978
w_2	0.2000	8.77×10^{-4}	0.19956
w_3	0.2500	4.98×10^{-4}	0.24975
w_4	0.3000	9.95×10^{-4}	0.29950

Error decay over training



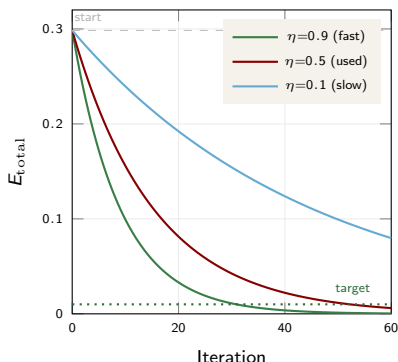
Key factors in training:

- Learning rate η : too large $\Rightarrow$ overshoot; too small $\Rightarrow$ slow
- Weight init: random near zero; breaks neuron symmetry
- Network **depth** & activation choice
- **Mini-batch** size

One-step summary

	Before	After	Change
E_{o_1}	0.2748	0.2468	-10.2%
E_{o_2}	0.0236	0.0210	-11.0%
E_{total}	0.2984	0.2678	-10.3%

Learning rate comparison

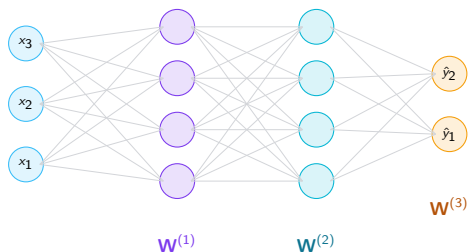


The Optimisation Problem

minimising empirical risk over training data

Goal: Find parameters θ minimising average loss over $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$:

$$\theta^* = \arg \min_{\theta} \underbrace{\frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(\mathbf{x}_i; \theta), y_i)}_{\text{empirical risk}}$$



The Optimisation Problem

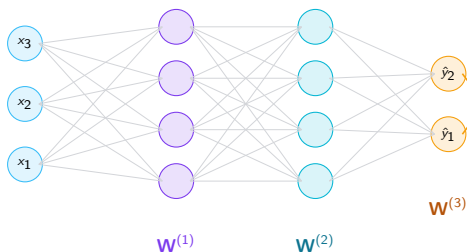
minimising empirical risk over training data

Goal: Find parameters θ minimising average loss over $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$:

$$\theta^* = \arg \min_{\theta} \underbrace{\frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(\mathbf{x}_i; \theta), y_i)}_{\text{empirical risk}}$$

Common loss functions:

- Cross-entropy: $\mathcal{L} = -\sum_c y_c \log \hat{p}_c$
- MSE: $\mathcal{L} = \frac{1}{2} \|\mathbf{y} - \hat{\mathbf{y}}\|^2$
- Cox: $\mathcal{L} = -\sum_i d_i [\eta_i - \log \sum_{j \in R_i} e^{\eta_j}]$



The Optimisation Problem

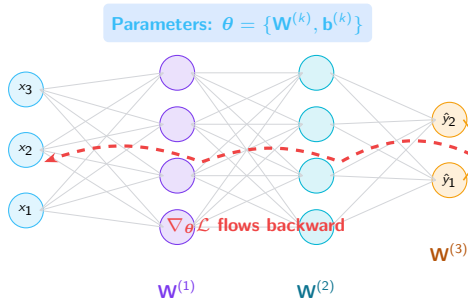
minimising empirical risk over training data

Goal: Find parameters θ minimising average loss over $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$:

$$\theta^* = \arg \min_{\theta} \underbrace{\frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(\mathbf{x}_i; \theta), y_i)}_{\text{empirical risk}}$$

Common loss functions:

- Cross-entropy: $\mathcal{L} = -\sum_c y_c \log \hat{p}_c$
- MSE: $\mathcal{L} = \frac{1}{2} \|\mathbf{y} - \hat{\mathbf{y}}\|^2$
- Cox: $\mathcal{L} = -\sum_i d_i [\eta_i - \log \sum_{j \in R_i} e^{\eta_j}]$



Why iterative methods? Deep networks have millions of parameters and *non-convex* loss surfaces with no analytical solution. We must use **gradient-based iteration**.

Gradient Descent: The Core Idea

animate: iterative descent — 8 pre-computed steps

Update rule:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t)$$

$\eta > 0$: learning rate. $\nabla \mathcal{L}$ points **uphill** — so we step **opposite** to it.

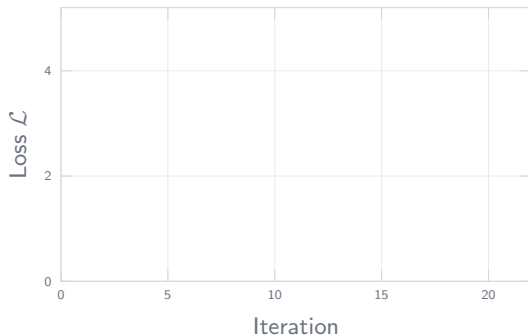
Tangent line at θ_t : slope = $\partial \mathcal{L} / \partial \theta$.

Arrow: the update step $-\eta \nabla \mathcal{L}$.

Convergence (convex loss): guaranteed when $\eta < 2/L$, where L is the Lipschitz constant[4] of the gradient. Here $L = 0.9$, so any $\eta < 2.2$ works.

Batch GD vs SGD vs Mini-batch

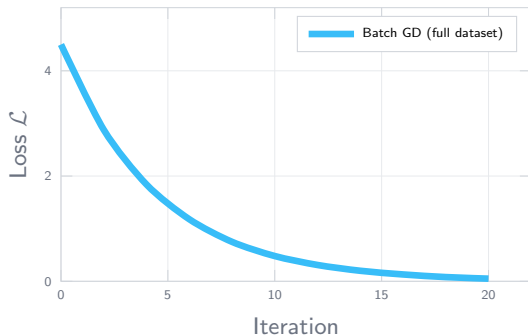
how many samples per gradient estimate?



Method	Gradient	Speed
Batch GD	Exact	Slow
SGD	Noisy	Fast
Mini-batch	Low noise	Best

Batch GD vs SGD vs Mini-batch

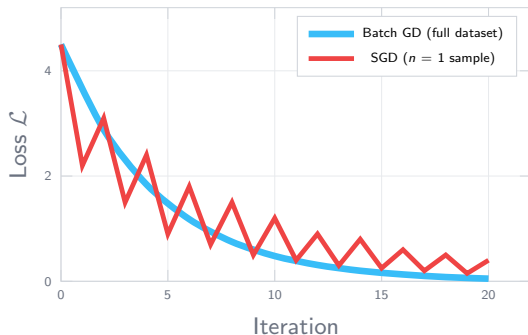
how many samples per gradient estimate?



Method	Gradient	Speed
Batch GD	Exact	Slow
SGD	Noisy	Fast
Mini-batch	Low noise	Best

Batch GD vs SGD vs Mini-batch

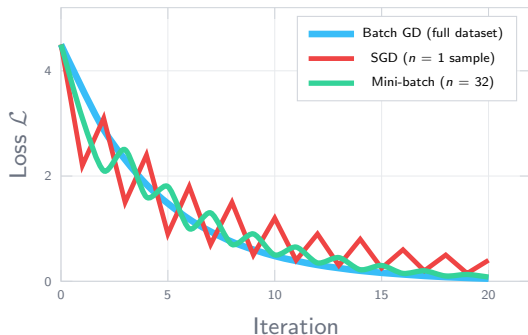
how many samples per gradient estimate?



Method	Gradient	Speed
Batch GD	Exact	Slow
SGD	Noisy	Fast
Mini-batch	Low noise	Best

Batch GD vs SGD vs Mini-batch

how many samples per gradient estimate?



Method	Gradient	Speed
Batch GD	Exact	Slow
SGD	Noisy	Fast
Mini-batch	Low noise	Best

In practice: mini-batch SGD ($n = 32$ – 256) is the **default** for training all modern ANNs — GPU-parallelism plus the regularising noise of SGD.

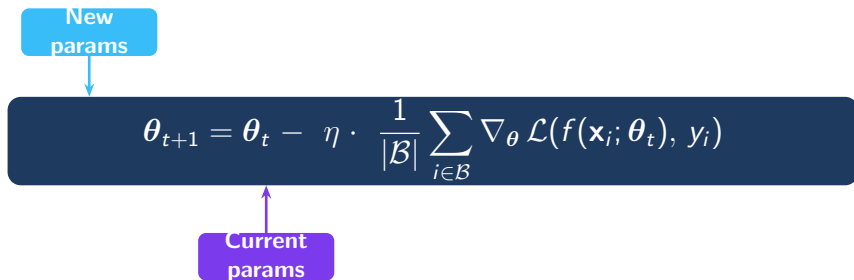
The SGD Update Rule

anatomy of the gradient step

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\theta} \mathcal{L}(f(\mathbf{x}_i; \theta_t), y_i)$$

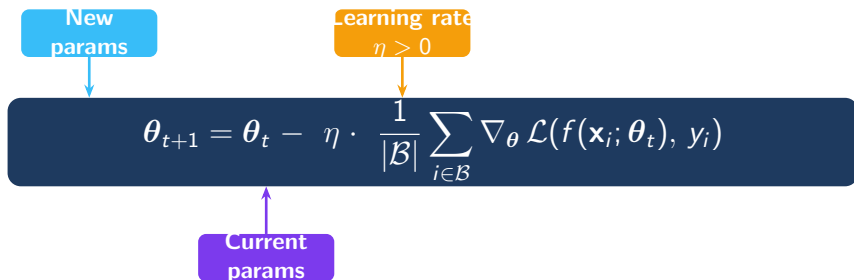
The SGD Update Rule

anatomy of the gradient step



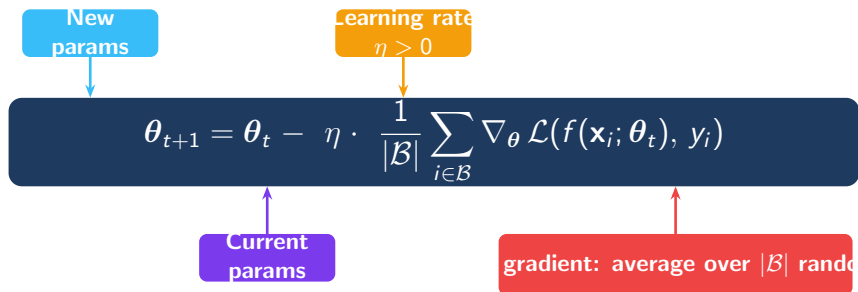
The SGD Update Rule

anatomy of the gradient step



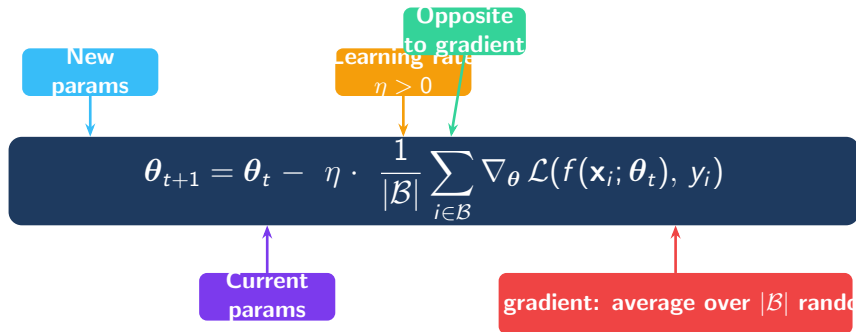
The SGD Update Rule

anatomy of the gradient step



The SGD Update Rule

anatomy of the gradient step



Learning rate η

Too large: diverges. Too small: very slow. Typical: 10^{-4} – 10^{-2} .

Mini-batch $\mathcal{B}$

Size 32–256. Larger batches $\Rightarrow$ smoother gradients but need proportionally larger η .

Descent direction

Negative gradient points toward the steepest local descent on the loss surface.

Effect of the Learning Rate

animate: too high diverges, too low stalls, good LR converges

High LR ($\eta = 2.10$)

Steps overshoot the minimum.
Update $\theta(1 - 2\eta)$ diverges when
 $\eta > 1$ for a quadratic loss.

Good LR ($\eta = 0.30$)

Converges in ≈ 8 steps. Satisfies
 $\eta < 2/L$ where $L = 0.9$.

Low LR ($\eta = 0.05$)

Correct direction but still at 58%
of initial loss after 20 steps —
very slow convergence.

SGD with Momentum

animate: noisy SGD vs smooth momentum trajectory

Momentum update:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + (1-\beta) \nabla \mathcal{L}_t$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{v}_{t+1}$$

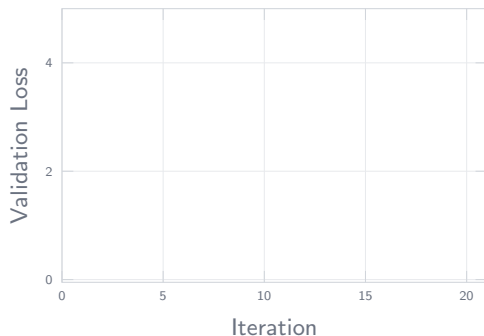
$\mathbf{v}_t$ is an exponential moving average of past gradients. $\beta=0.9$ typical.

Benefits:

- Damps gradient oscillations
- Accelerates descent in consistent direction
- Helps escape shallow local minima
- Effective LR $\approx \eta/(1-\beta)$

Adaptive Learning Rate Methods

RMSprop, Adam, AdamW — per-parameter adaptation



AdamW: Adam with *decoupled* weight decay. Preferred for Transformers. Use $\lambda = 0.01$ – 0.1 .

RMSprop [10]:

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t, \quad v_t = \rho v_{t-1} + (1 - \rho) g_t^2$$

Adapts LR per-parameter by its gradient variance.

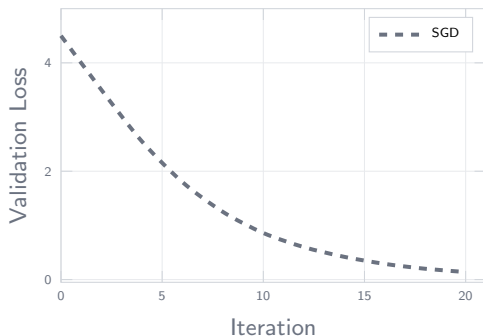
Adam [5]: Combines momentum (m_t) and RMSprop (v_t):

$$\theta_t \leftarrow \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

$$\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$$

Adaptive Learning Rate Methods

RMSprop, Adam, AdamW — per-parameter adaptation



AdamW: Adam with *decoupled* weight decay. Preferred for Transformers. Use $\lambda = 0.01$ – 0.1 .

RMSprop [10]:

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t, \quad v_t = \rho v_{t-1} + (1 - \rho) g_t^2$$

Adapts LR per-parameter by its gradient variance.

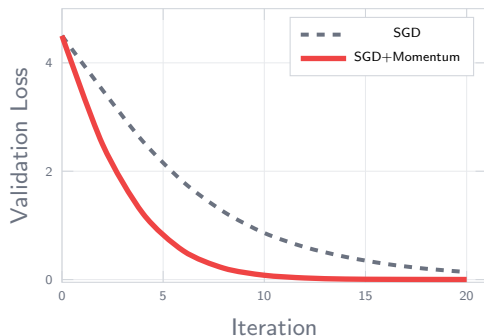
Adam [5]: Combines momentum (m_t) and RMSprop (v_t):

$$\theta_t \leftarrow \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

$$\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$$

Adaptive Learning Rate Methods

RMSprop, Adam, AdamW — per-parameter adaptation



AdamW: Adam with *decoupled* weight decay. Preferred for Transformers. Use $\lambda = 0.01$ – 0.1 .

RMSprop [10]:

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t, \quad v_t = \rho v_{t-1} + (1 - \rho) g_t^2$$

Adapts LR per-parameter by its gradient variance.

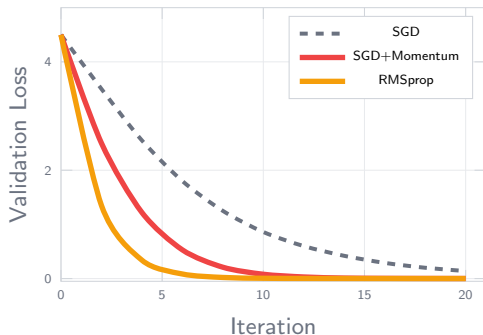
Adam [5]: Combines momentum (m_t) and RMSprop (v_t):

$$\theta_t \leftarrow \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

$$\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$$

Adaptive Learning Rate Methods

RMSprop, Adam, AdamW — per-parameter adaptation



AdamW: Adam with *decoupled* weight decay. Preferred for Transformers. Use $\lambda = 0.01$ – 0.1 .

RMSprop [10]:

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t, \quad v_t = \rho v_{t-1} + (1 - \rho) g_t^2$$

Adapts LR per-parameter by its gradient variance.

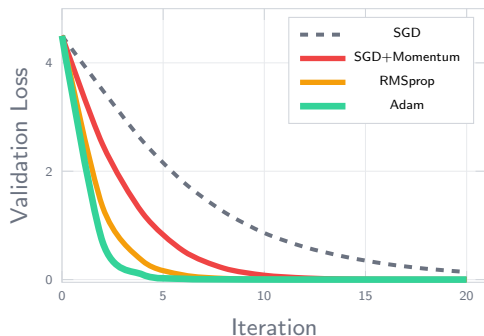
Adam [5]: Combines momentum (m_t) and RMSprop (v_t):

$$\theta_t \leftarrow \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

$$\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$$

Adaptive Learning Rate Methods

RMSprop, Adam, AdamW — per-parameter adaptation



AdamW: Adam with *decoupled* weight decay. Preferred for Transformers. Use $\lambda = 0.01$ – 0.1 .

RMSprop [10]:

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t, \quad v_t = \rho v_{t-1} + (1 - \rho) g_t^2$$

Adapts LR per-parameter by its gradient variance.

Adam [5]: Combines momentum (m_t) and RMSprop (v_t):

$$\theta_t \leftarrow \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

$$\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$$

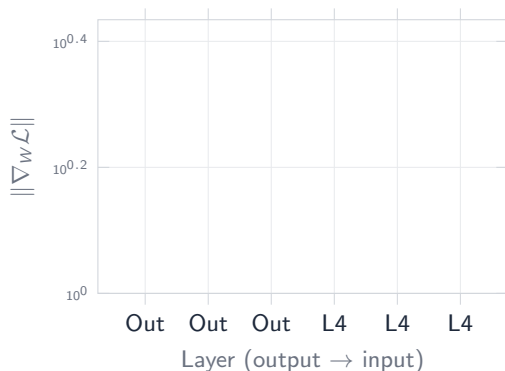
Backpropagation: Computing the Gradient

animate: forward pass then backward gradient flow

Cost: One forward+backward $\approx 2 \times$ the Floating Point Operations (FLOPs) of inference alone. All layer gradients computed in a single backward pass.

Vanishing and Exploding Gradients

pgfplots: gradient magnitude per layer depth



Vanishing: Sigmoid/Tanh saturate $\Rightarrow$ derivative ≈ 0 . Gradients shrink exponentially with depth. Early layers receive essentially zero signal.

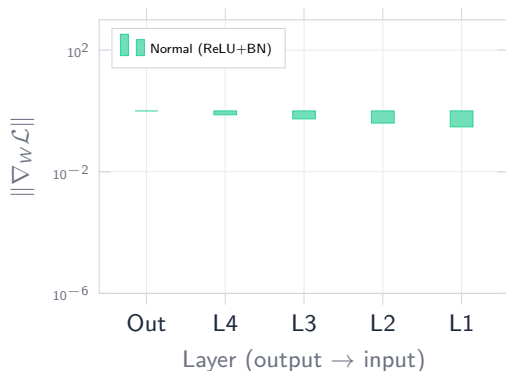
Exploding: Large weights amplify gradients backward. Training diverges (NaN loss). Common in deep RNNs on long sequences.

Solutions:

- **ReLU** activations (no saturation)
- **Batch Normalisation** after each layer

Vanishing and Exploding Gradients

pgplots: gradient magnitude per layer depth



Vanishing: Sigmoid/Tanh saturate $\Rightarrow$ derivative ≈ 0 . Gradients shrink exponentially with depth. Early layers receive essentially zero signal.

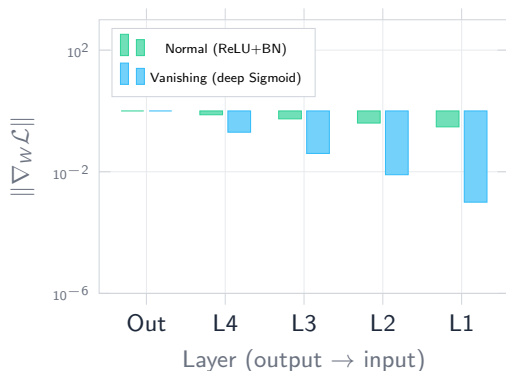
Exploding: Large weights amplify gradients backward. Training diverges (NaN loss). Common in deep RNNs on long sequences.

Solutions:

- **ReLU** activations (no saturation)
- **Batch Normalisation** after each layer

Vanishing and Exploding Gradients

pgfplots: gradient magnitude per layer depth



Vanishing: Sigmoid/Tanh saturate $\Rightarrow$ derivative ≈ 0 . Gradients shrink exponentially with depth. Early layers receive essentially zero signal.

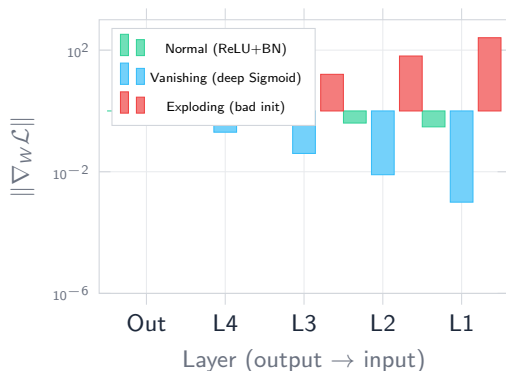
Exploding: Large weights amplify gradients backward. Training diverges (NaN loss). Common in deep RNNs on long sequences.

Solutions:

- **ReLU** activations (no saturation)
- **Batch Normalisation** after each layer

Vanishing and Exploding Gradients

pgfplots: gradient magnitude per layer depth



Vanishing: Sigmoid/Tanh saturate $\Rightarrow$ derivative ≈ 0 . Gradients shrink exponentially with depth. Early layers receive essentially zero signal.

Exploding: Large weights amplify gradients backward. Training diverges (NaN loss). Common in deep RNNs on long sequences.

Solutions:

- **ReLU** activations (no saturation)
- **Batch Normalisation** after each layer

Practical Guidelines for SGD Optimisation

a decision framework for practitioners

Optimiser Selection Guide

Small dataset ($n < 1000$)

Use: SGD+Momentum, LR = 10^{-2}

Large dataset; vision tasks

Use: Adam or AdamW, LR = 10^{-3}

NLP / Transformers

Use: AdamW + cosine LR schedule

Fine-tuning pretrained

Use: Adam, small LR = 10^{-5}

Interpretability required

Use: SGD (no adaptive scaling)

Debugging loss curves: Both flat $\Rightarrow$ LR too low or saddle point. Val $\gg$ Train $\Rightarrow$ overfitting. Train $\gg$ Val $\Rightarrow$ underfitting.

Learning rate schedules:

Warm-up: increase from 0 over first k steps

Key hyperparameter defaults:

- LR: 10^{-3} (Adam), 10^{-2} (SGD)
- Batch size: 32–128 for most tasks
- Weight decay: 10^{-4} – 10^{-2} (AdamW)
- Momentum: $\beta_1 = 0.9$
- Gradient clipping: $c = 1.0$ for RNNs/Transformers

Update Rule

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

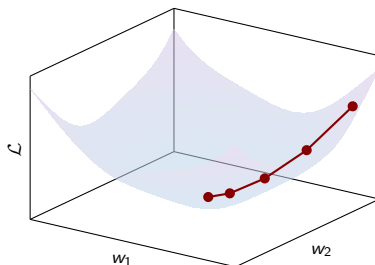
Variants:

- **Batch GD** — full dataset; exact, slow
- **SGD** — one sample; noisy, fast
- **Mini-batch SGD** — best of both (typical: 32–256)

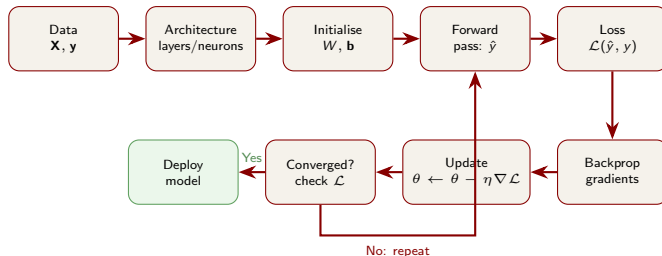
Advanced optimisers:

- **Momentum:** $v \leftarrow \beta v - \eta \nabla \mathcal{L}$
- **RMSProp:** adaptive per-parameter η
- **Adam:** momentum + adaptive

Loss Surface & Descent Path



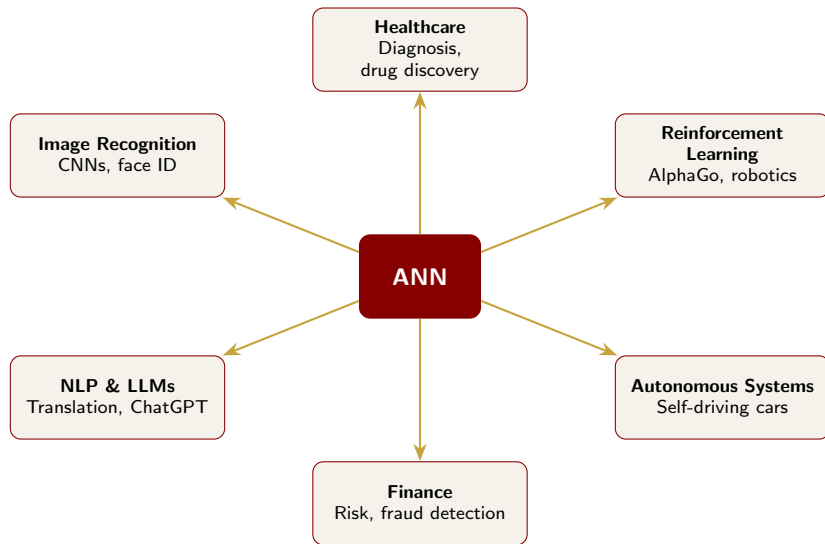
Summary — The ANN Training Pipeline



Data → **Architecture** → **Forward Pass** → **Loss** → **Backprop** → **Update** → **Repeat**

Computational cost

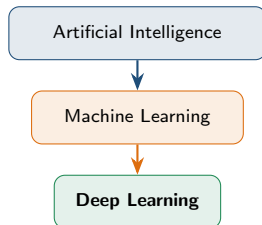
Both passes: $\mathcal{O}(|\text{weights}|)$ — **linear** in network size. This efficiency makes deep learning tractable.



- 1 AI Overview
 - Intelligence
 - AI landscape
 - ML Milestones
- 2 Foundations of AI
- 3 Artificial Neural Networks
 - Learning and loss function
- 4 Deep Learning
- 5 Transfer Learning
- 6 PAMHOYA Discovery Platform

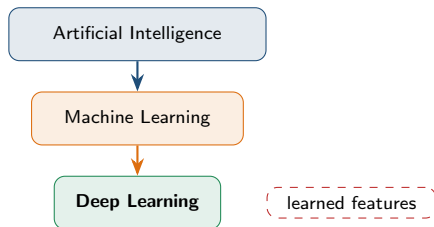
What is Deep Learning?

- Subfield of machine learning using multi-layer neural networks



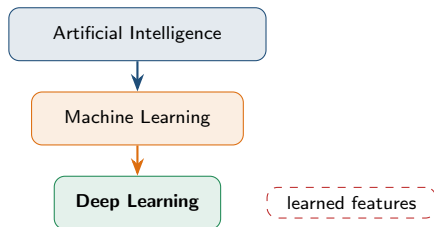
What is Deep Learning?

- Subfield of machine learning using **multi-layer neural networks**
- Learns hierarchical representations *directly from data* — no hand-crafted features



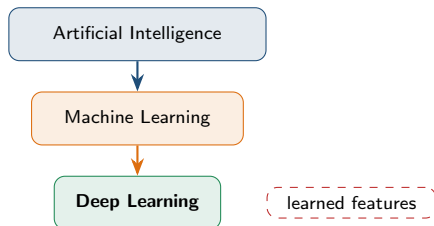
What is Deep Learning?

- Subfield of machine learning using **multi-layer neural networks**
- Learns hierarchical representations *directly from data* — no hand-crafted features
- Powers vision, speech, NLP, generative AI



What is Deep Learning?

- Subfield of machine learning using **multi-layer neural networks**
- Learns hierarchical representations *directly from data* — no hand-crafted features
- Powers vision, speech, NLP, generative AI
- Driven by: big data, GPUs, algorithmic advances [3]



Convolutional Neural Networks (CNNs)

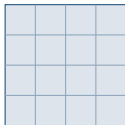
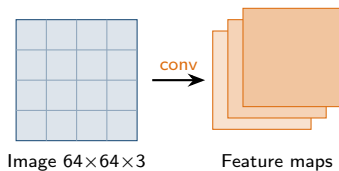


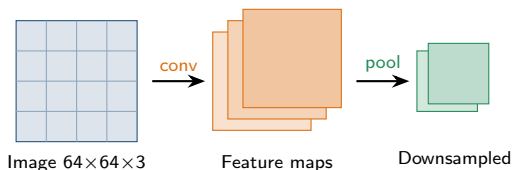
Image $64 \times 64 \times 3$

Convolutional Neural Networks (CNNs)



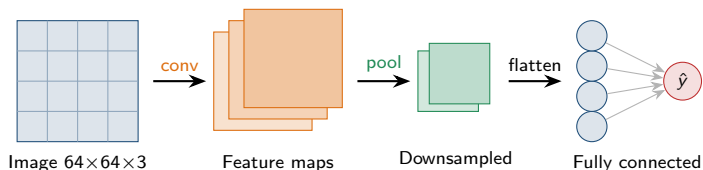
- **Convolution:** shared filters detect local patterns (edges $\rightarrow$ textures $\rightarrow$ objects)

Convolutional Neural Networks (CNNs)



- **Convolution:** shared filters detect local patterns (edges $\rightarrow$ textures $\rightarrow$ objects)
- **Pooling:** downsamples, adds translation invariance

Convolutional Neural Networks (CNNs)



- **Convolution**: shared filters detect local patterns (edges $\rightarrow$ textures $\rightarrow$ objects)
- **Pooling**: downsamples, adds translation invariance
- Far fewer parameters than dense layers $\Rightarrow$ efficient on images

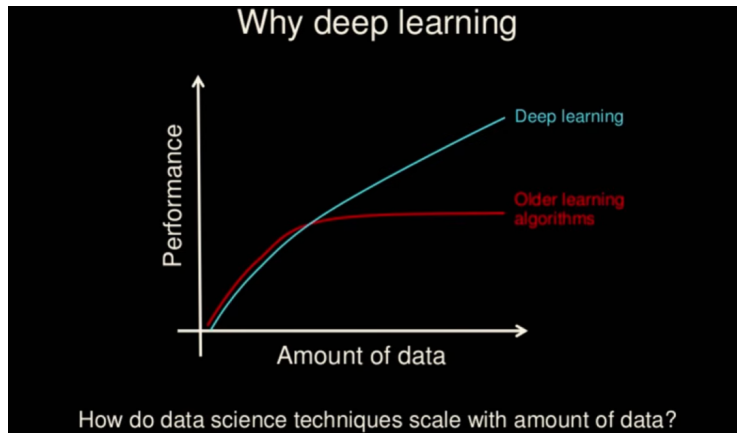


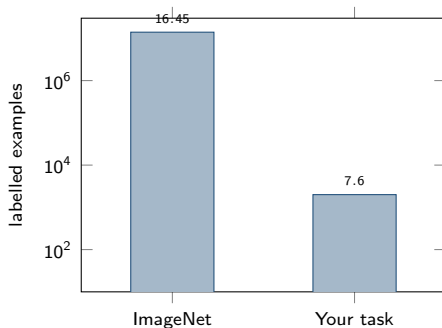
Figure: Scalable learning

- 1 AI Overview
 - Intelligence
 - AI landscape
 - ML Milestones
- 2 Foundations of AI
- 3 Artificial Neural Networks
 - Learning and loss function
- 4 Deep Learning
- 5 Transfer Learning
- 6 PAMHOYA Discovery Platform



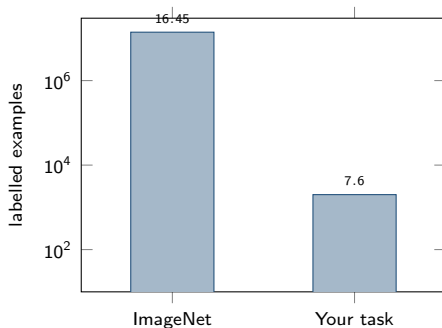
Why Transfer Learning?

- Deep networks are data-hungry:
millions of labelled examples



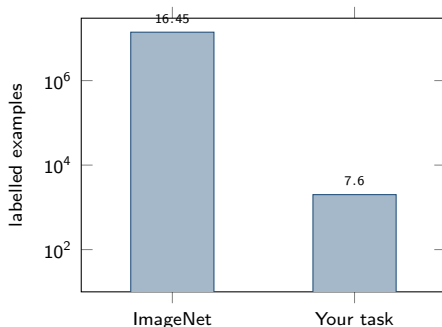
Why Transfer Learning?

- Deep networks are data-hungry: millions of labelled examples
- Most real tasks have **small labelled datasets** (medical imaging, niche domains)



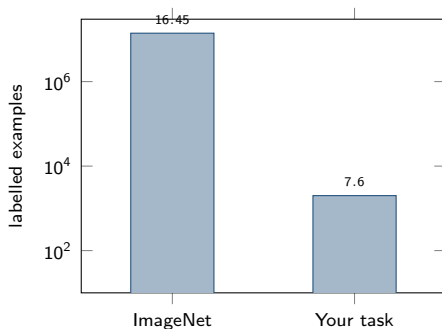
Why Transfer Learning?

- Deep networks are data-hungry: millions of labelled examples
- Most real tasks have **small labelled datasets** (medical imaging, niche domains)
- Labelling is expensive; *related* large datasets often exist



Why Transfer Learning?

- Deep networks are data-hungry: millions of labelled examples
- Most real tasks have **small labelled datasets** (medical imaging, niche domains)
- Labelling is expensive; *related* large datasets often exist
- Idea: **reuse** what a model learned on a big source task



4 orders of magnitude of free supervision to exploit.

"Deep learning"



Human learning



Representations

Models

Skills

Cinema mode T

Figure: Knowledge transfer in humans

Domain

$D = \{X; P(X)\}$ — a feature space X and a marginal distribution $P(X)$, $X \in \mathcal{X}$.

Domain

$D = \{X; P(X)\}$ — a feature space X and a marginal distribution $P(X)$, $X \in \mathcal{X}$.

Task

$T = \{Y; f(\cdot)\}$ — a label space Y and a predictive function
 $f(x) \approx P(Y \mid X = x)$.

Domain

$D = \{X; P(X)\}$ — a feature space X and a marginal distribution $P(X)$, $X \in \mathcal{X}$.

Task

$T = \{Y; f(\cdot)\}$ — a label space Y and a predictive function
 $f(x) \approx P(Y \mid X = x)$.

Source

$D_S = \{X_S, P_S(X)\}$ $T_S = \{Y_S, f_S\}$ n_S large

transfer

Target

$D_T = \{X_T, P_T(X)\}$ $T_T = \{Y_T, f_T\}$, n_T small

Transfer Learning

Given source (D_S, T_S) and target (D_T, T_T) with

$$D_S \neq D_T \quad \text{or} \quad T_S \neq T_T,$$

transfer learning aims to improve the target predictor $f_T(\cdot)$ using knowledge from D_S and T_S . [7]

Transfer Learning

Given source (D_S, T_S) and target (D_T, T_T) with

$$D_S \neq D_T \quad \text{or} \quad T_S \neq T_T,$$

transfer learning aims to improve the target predictor $f_T(\cdot)$ using knowledge from D_S and T_S . [7]

What can differ?

Transfer Learning

Given source (D_S, T_S) and target (D_T, T_T) with

$$D_S \neq D_T \quad \text{or} \quad T_S \neq T_T,$$

transfer learning aims to improve the target predictor $f_T(\cdot)$ using knowledge from D_S and T_S . [7]

What can differ? Each mismatch defines a sub-problem:

- $X_S \neq X_T$: different feature spaces (heterogeneous transfer)

Transfer Learning

Given source (D_S, T_S) and target (D_T, T_T) with

$$D_S \neq D_T \quad \text{or} \quad T_S \neq T_T,$$

transfer learning aims to improve the target predictor $f_T(\cdot)$ using knowledge from D_S and T_S . [7]

What can differ? Each mismatch defines a sub-problem:

- $X_S \neq X_T$: different feature spaces (heterogeneous transfer)
- $P_S(X) \neq P_T(X)$: same features, shifted inputs (**domain adaptation** / covariate shift)

Transfer Learning

Given source (D_S, T_S) and target (D_T, T_T) with

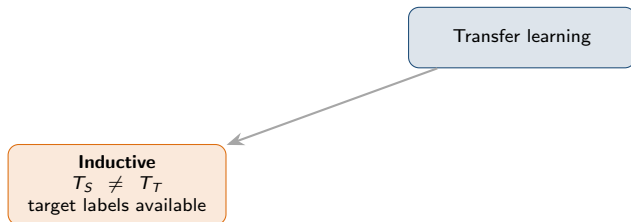
$$D_S \neq D_T \quad \text{or} \quad T_S \neq T_T,$$

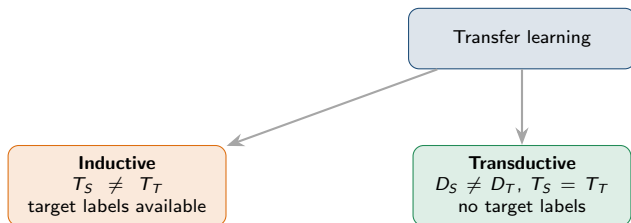
transfer learning aims to improve the target predictor $f_T(\cdot)$ using knowledge from D_S and T_S . [7]

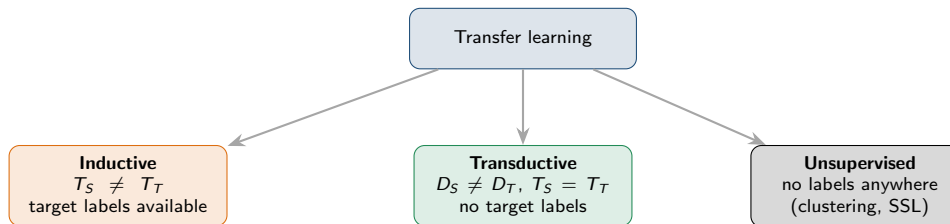
What can differ? Each mismatch defines a sub-problem:

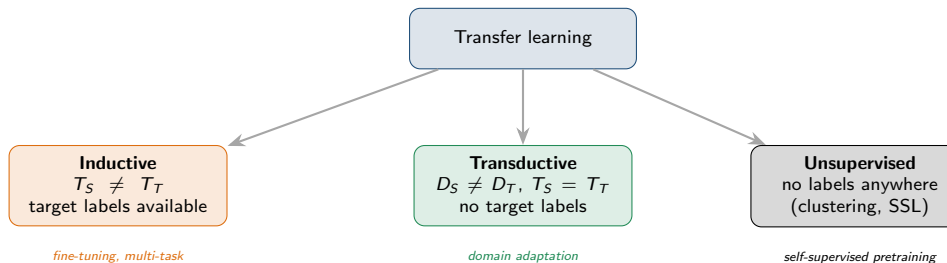
- $X_S \neq X_T$: different feature spaces (heterogeneous transfer)
- $P_S(X) \neq P_T(X)$: same features, shifted inputs (**domain adaptation** / covariate shift)
- $Y_S \neq Y_T$ or $P_S(Y|X) \neq P_T(Y|X)$: different tasks (**inductive transfer**, e.g. fine-tuning)

Transfer learning

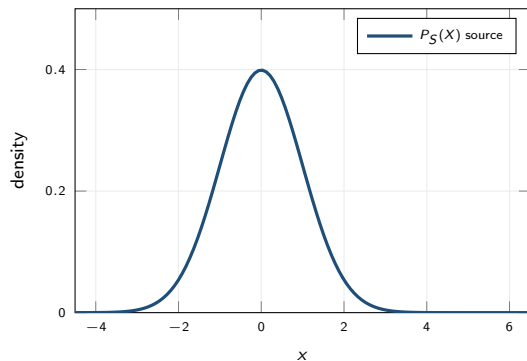






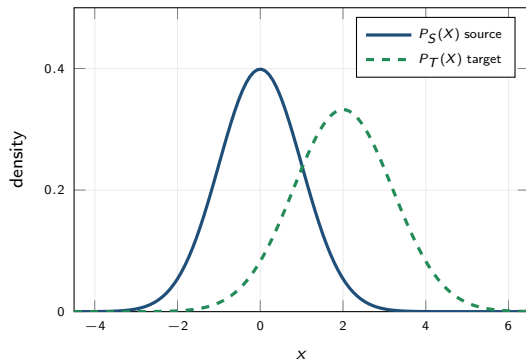


Distribution Shift, Visualised



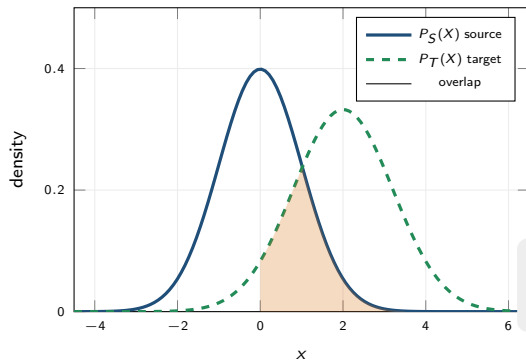
- Source model sees inputs $\sim P_S(X)$

Distribution Shift, Visualised



- Source model sees inputs $\sim P_S(X)$
- Deployment inputs $\sim P_T(X)$ are shifted

Distribution Shift, Visualised



- Source model sees inputs $\sim P_S(X)$
- Deployment inputs $\sim P_T(X)$ are shifted
- **Overlap** is where source knowledge is directly reusable

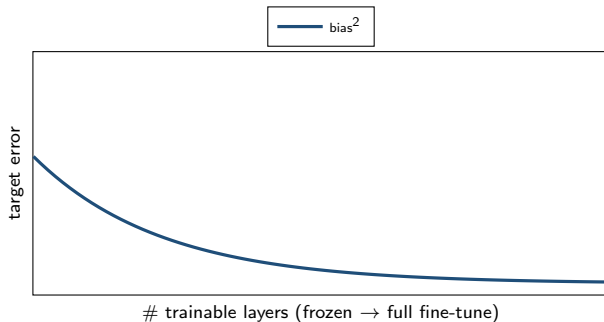
Transfer works when distributions share *structure* — in raw space or in a learned feature space.

Freezing pretrained layers trades **variance** for **bias**:

$$\mathbb{E}[(\hat{f}_T(x) - f_T(x))^2] = \underbrace{\text{Bias}^2}_{\substack{\text{source features may} \\ \text{not suit target}}} + \underbrace{\text{Variance}}_{\substack{\text{fewer trainable params} \\ \Rightarrow \text{much smaller}}} + \sigma^2$$

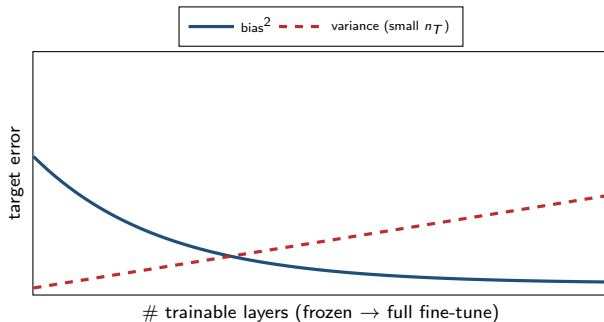
Freezing pretrained layers trades **variance** for **bias**:

$$\mathbb{E}[(\hat{f}_T(x) - f_T(x))^2] = \underbrace{\text{Bias}^2}_{\substack{\text{source features may} \\ \text{not suit target}}} + \underbrace{\text{Variance}}_{\substack{\text{fewer trainable params} \\ \Rightarrow \text{much smaller}}} + \sigma^2$$



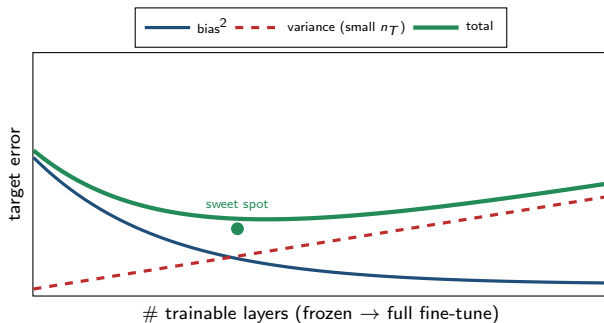
Freezing pretrained layers trades **variance** for **bias**:

$$\mathbb{E}[(\hat{f}_T(x) - f_T(x))^2] = \underbrace{\text{Bias}^2}_{\substack{\text{source features may} \\ \text{not suit target}}} + \underbrace{\text{Variance}}_{\substack{\text{fewer trainable params} \\ \Rightarrow \text{much smaller}}} + \sigma^2$$



Freezing pretrained layers trades **variance** for **bias**:

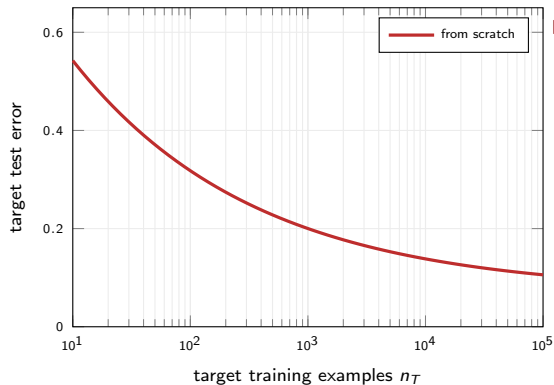
$$\mathbb{E}[(\hat{f}_T(x) - f_T(x))^2] = \underbrace{\text{Bias}^2}_{\substack{\text{source features may} \\ \text{not suit target}}} + \underbrace{\text{Variance}}_{\substack{\text{fewer trainable params} \\ \Rightarrow \text{much smaller}}} + \sigma^2$$



Transfer success

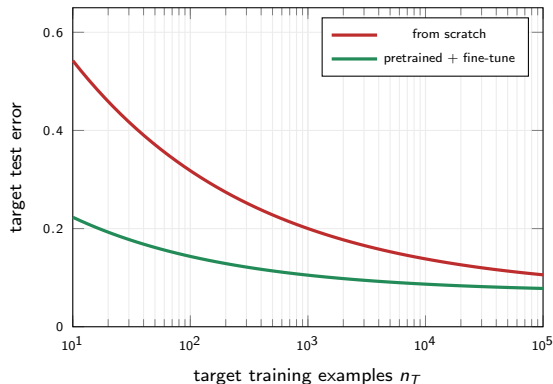
Transfer succeeds $\iff$ source is well-learned, domains look alike *in feature space*, and one predictor can serve both. Representation learning attacks the middle term.

The Payoff: Learning Curves



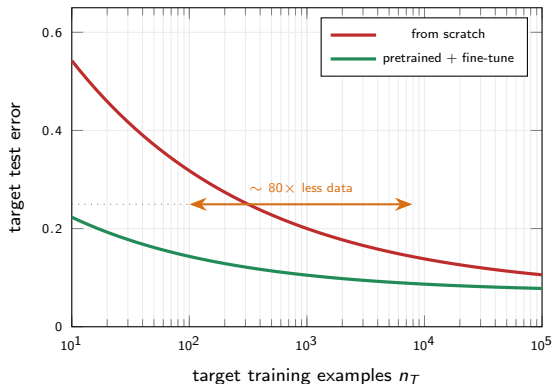
- From scratch: error decays slowly with n_T

The Payoff: Learning Curves



- From scratch: error decays slowly with n_T
- Pretraining shifts the whole curve down: **better start, better asymptote in the low-data regime**

The Payoff: Learning Curves

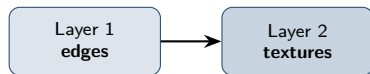


- From scratch: error decays slowly with n_T
- Pretraining shifts the whole curve down: **better start, better asymptote in the low-data regime**
- Same accuracy with orders of magnitude fewer labels

Intuition: Features Are Reusable Because They're Hierarchical

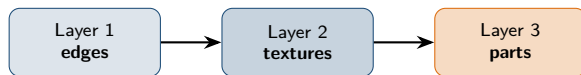
Layer 1
edges

Intuition: Features Are Reusable Because They're Hierarchical



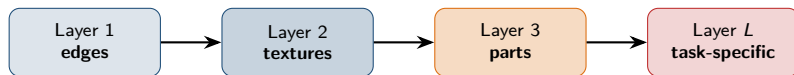
- Early layers learn statistics of *natural data* — nearly universal

Intuition: Features Are Reusable Because They're Hierarchical



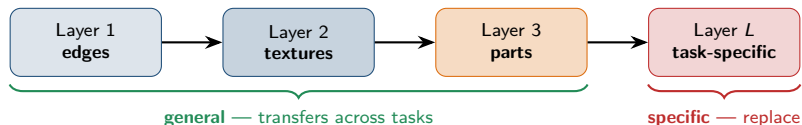
- Early layers learn statistics of *natural data* — nearly universal

Intuition: Features Are Reusable Because They're Hierarchical



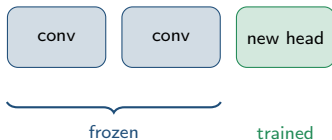
- Early layers learn statistics of *natural data* — nearly universal
- Specificity grows with depth [11]

Intuition: Features Are Reusable Because They're Hierarchical



- Early layers learn statistics of *natural data* — nearly universal
- Specificity grows with depth [11]
- Hence the recipe: **keep the bottom, retrain the top**

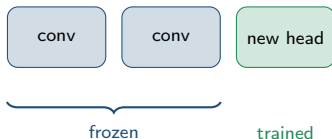
A. Feature extraction



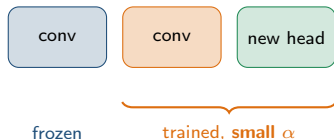
- **A:** treat the network as a fixed feature map $\phi(x)$; train a light classifier on $\phi(x)$ — best for tiny n_T

Strategies: Feature Extraction vs. Fine-Tuning

A. Feature extraction



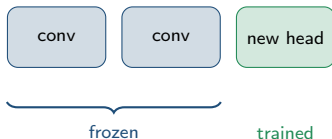
B. Fine-tuning



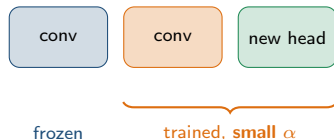
- **A:** treat the network as a fixed feature map $\phi(x)$; train a light classifier on $\phi(x)$ — best for tiny n_T
- **B:** unfreeze top layers with a **small learning rate** — best when n_T is moderate

Strategies: Feature Extraction vs. Fine-Tuning

A. Feature extraction

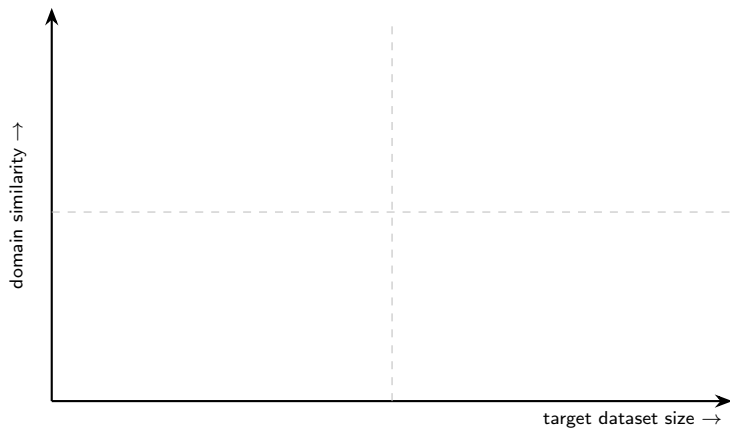


B. Fine-tuning

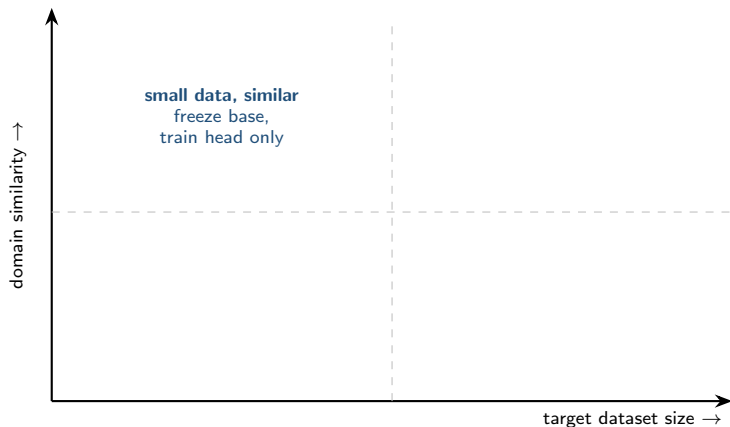


- **A:** treat the network as a fixed feature map $\phi(x)$; train a light classifier on $\phi(x)$ — best for tiny n_T
- **B:** unfreeze top layers with a **small learning rate** — best when n_T is moderate
- Rule of thumb: *the more target data, the deeper you fine-tune*

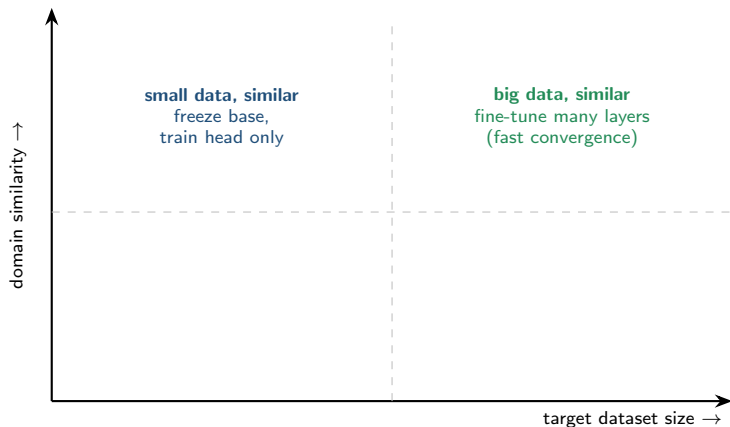
Choosing a Strategy



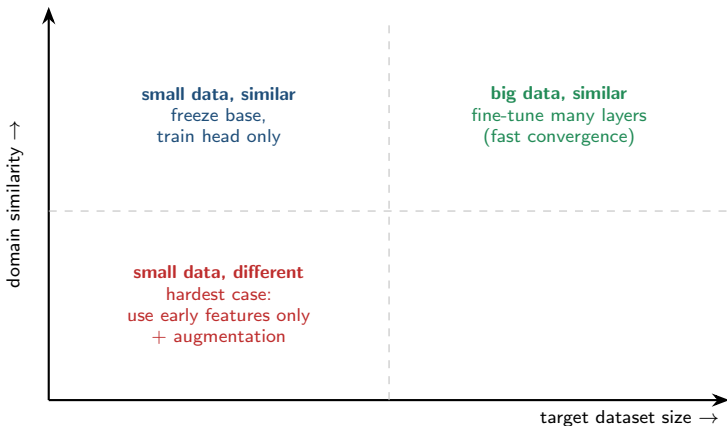
Choosing a Strategy



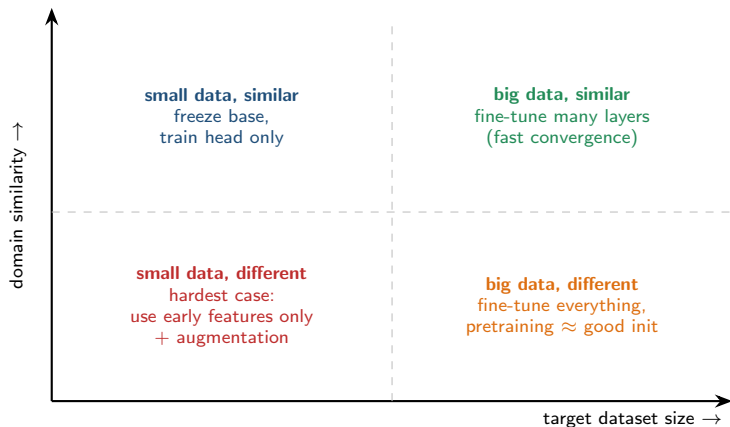
Choosing a Strategy



Choosing a Strategy



Choosing a Strategy



Built on Foundation Models

large language & vision models derived from foundation models

Language

EraDGPT-3 (2020)

- ↪ **InstructGPT** — RLHF fine-tune
- ↪ **ChatGPT** — chat-tuned (SFT+RLHF)

EraDLaMA / Llama 2

- ↪ **Alpaca** — instruction SFT
- ↪ **Vicuna** — chat SFT (ShareGPT)

EraCBERT (2018)

- ↪ **RoBERTa** — retrained, more data
- ↪ **DistilBERT** — knowledge distillation

Vision / Multimodal

EraDCLIP (2021)

- ↪ **Stable Diffusion** — reuses CLIP encoder
- ↪ **LLaVA** — CLIP encoder + LLM

EraDStable Diffusion

- ↪ **DreamBooth** — subject fine-tune
- ↪ **ControlNet** — conditioning adapter

EraCViT (2020)

- ↪ **DeiT** — distillation training
- ↪ **DINO** — self-supervised variant

Pattern: pretrain once on internet-scale data, then
adapt via fine-tuning, distillation, or lightweight adapters — transfer learning at the
largest scale.

Portable Models via Transfer Learning

compressing foundation models for mobile & edge deployment

Model	Derived From	Technique	Portability Gain
Language			
DistilBERT	BERT	Distillation	40% smaller, 60% faster
TinyBERT	BERT	2-stage distillation	7.5× smaller, 9.4× faster
MobileBERT	BERT-large	Bottleneck distillation	4.3× smaller, 5.5× faster
Vision / Multimodal			
MobileNet	ImageNet CNNs	Depthwise-sep. convs	~30× fewer FLOPs vs. VGG
MobileViT	ViT	Hybrid CNN+Transformer	<6M parameters
MobileCLIP	CLIP	Multi-teacher distillation	Up to 10× smaller, real-time

Pattern: every portable model still starts from a large pretrained teacher —
distillation, pruning, quantization, and architecture redesign are all ways of transferring that knowledge onto a phone.

TB Screening (Xrays) Case study: Transfer Learning



Figure: Sample X-rays

```
base_model <- application_mobilenet_v2(  
  input_shape = c(img_size, 3L),  
  include_top = FALSE,  
  weights = "imagenet"  
)  
freeze_weights(base_model)  
# preprocessing -> frozen base -> classification head  
inputs <- layer_input(shape = c(img_size, 3L))  
  
preprocessed <- inputs |>  
  layer_rescaling(scale = 1/255) |>  
  layer_random_flip("horizontal") |>  
  layer_random_rotation(factor = 0.028) |>  
  layer_random_zoom(height_factor = 0.1) |>  
  layer_random_translation(height_factor = 0.05, width_factor = 0.05)
```

Figure: Model setup

Data

- train=2940 files belonging to 2 classes.
- validation=630 files for validation
- test=630 files belonging to 2 classes.
- class balancing: class weights

TB Screening (Xrays) Case study: Transfer Learning

```
output <- base_out |>
  layer_global_average_pooling_2d() |>
  layer_dropout(rate = 0.3) |>
  layer_dense(units = 128, activation = "relu") |>
  layer_batch_normalization() |>
  layer_dropout(rate = 0.2) |>
  layer_dense(units = 1, activation = "sigmoid")

model <- keras_model(inputs = inputs, outputs = output)

model |> compile(
  optimizer = optimiser_adam(learning_rate = 1e-3),
  loss = "binary_crossentropy",
  metrics = list("accuracy", metric_auc(name = "auc"))
)
```

Figure: Learning Optimization

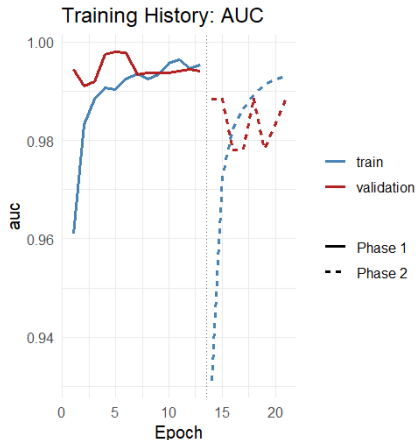
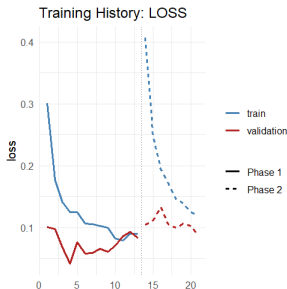


Figure: Training Performance AUC

TB Screening (Xrays) Case study: Transfer Learning

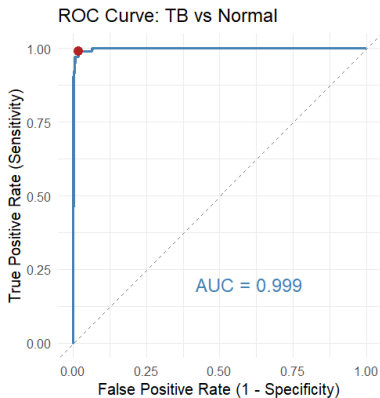


Figure: Test Performance

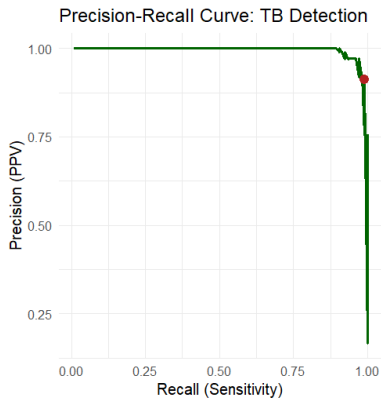


Figure: Precision recall curve

Core ideas

- 1 Transfer = reuse knowledge when $D_S \neq D_T$ or $T_S \neq T_T$
- 2 Works because it cuts **sample complexity** and **variance**
- 3 Ben-David bound [1, 8]: success = low source error + aligned domains + compatible tasks
- 4 Hierarchical features: general at the bottom, specific at the top

Practical recipe

- 1 Start from a strong pretrained model
- 2 Replace the head; freeze the base
- 3 Unfreeze top layers as data allows; use a small learning rate
- 4 Watch the from-scratch baseline (negative transfer!)

Core ideas

- 1 Transfer = reuse knowledge when $D_S \neq D_T$ or $T_S \neq T_T$
- 2 Works because it cuts **sample complexity** and **variance**
- 3 Ben-David bound [1, 8]: success = low source error + aligned domains + compatible tasks
- 4 Hierarchical features: general at the bottom, specific at the top

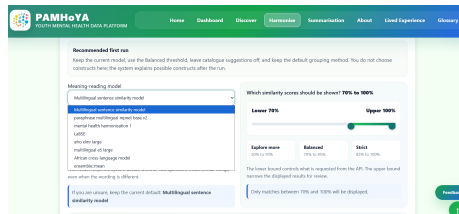
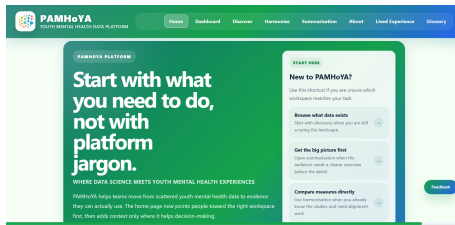
Practical recipe

- 1 Start from a strong pretrained model
- 2 Replace the head; freeze the base
- 3 Unfreeze top layers as data allows; use a small learning rate
- 4 Watch the from-scratch baseline (negative transfer!)



- 1 AI Overview
 - Intelligence
 - AI landscape
 - ML Milestones
- 2 Foundations of AI
- 3 Artificial Neural Networks
 - Learning and loss function
- 4 Deep Learning
- 5 Transfer Learning
- 6 PAMHOYA Discovery Platform

PAMHOYA Discovery & Harmonization Platform



Team

The PAMYOYA Team

Key Message

- Data quality is important for optimal AI solutions
- Methods and process literacy is important for interaction between teams for successful ML/AI solutions
- Adapting/adopting foundational models for AI solutions in resource limited countries possible
- Find your team and contribute to Healthcare solutions using SL/ML/AI methods

- [1] Shai Ben-David et al. “Analysis of representations for domain adaptation”. In: *Advances in neural information processing systems* 19 (2006).
- [2] John Burden and José Hernández-Orallo. “Exploring AI safety in degrees: generality, capability and control”. In: *Proceedings of the workshop on artificial intelligence safety (safeai 2020) co-located with 34th AAAI conference on artificial intelligence (AAAI 2020)*. ceur-ws. org. 2020, pp. 36–40.
- [3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [4] Matt Jordan and Alexandros G Dimakis. “Exactly computing the local lipschitz constant of relu networks”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 7344–7353.
- [5] Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980* (2014).

- [6] Rui Mao et al. “Bridging Minds and Machines: Toward an Integration of AI and Cognitive Science”. In: *arXiv preprint arXiv:2508.20674* (2025).
- [7] Sinno Jialin Pan and Qiang Yang. “A Survey on Transfer Learning”. In: *IEEE Transactions on Knowledge and Data Engineering* 22.10 (2010), pp. 1345–1359. DOI: 10.1109/TKDE.2009.191.
- [8] Ievgen Redko et al. “A survey on domain adaptation theory: learning bounds and theoretical guarantees”. In: *arXiv preprint arXiv:2004.11829* (2020).
- [9] Russell Stuart and Norvig Peter. *Artificial intelligence a modern approach third edition*. 2010.
- [10] T Tieleman and G Hinton. “Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude”. In: *COURSERA: Neural networks for machine learning* 4.2 (2012), p. 26.
- [11] Jason Yosinski et al. “How transferable are features in deep neural networks?” In: *Advances in neural information processing systems* 27 (2014).